

THredis: Tomasulo-Style Reservation Stations for Multi-Threaded Key-Value Stores

Le-Gao Chen*
legaoche@usc.edu
University of Southern California
Los Angeles, California, USA

Cunwei Chen
cunweich@usc.edu
University of Southern California
Los Angeles, California, USA

Harsh Rajesh Shah
hrshah@usc.edu
University of Southern California
Los Angeles, California, USA

Kaiwen Zheng
kzheng00@usc.edu
University of Southern California
Los Angeles, California, USA

Shahin Nazarian
shahin.nazarian@usc.edu
University of Southern California
Los Angeles, California, USA

Abstract

Redis’s single-threaded command execution model serializes all in-flight commands behind one execution thread, leaving multicore parallelism unexploited even when clients pipeline many requests on the same connection. Existing multi-threaded Redis-compatible systems such as KeyDB and DragonflyDB add parallelism across clients or shards but still treat pipelining as a wire-level batching mechanism rather than an opportunity to execute commands from the same connection concurrently.

We present THredis, a Redis-compatible in-memory key-value store that introduces *execution-level pipelining*: commands from the same connection may execute concurrently on independent worker threads while preserving Redis’s client-visible reply order and same-key semantics. THredis’s architecture mirrors Tomasulo’s out-of-order execution algorithm. IO threads dispatch parsed commands as *fake clients* through lock-free SPSC queues to hash-partitioned worker threads that own disjoint shards of the keyspace; workers signal completion through an atomic reply-ready bitmask; and an in-order drain retires replies in dispatch order. The single-writer-per-key invariant removes locks from the hot path, and the consolidated drain reduces send-side syscalls to one per pipelined batch rather than one per request.

We evaluate THredis against Redis, KeyDB, and DragonflyDB across three load generators (memtier_benchmark, redis-benchmark, YCSB) with sweeps over thread count, payload size, pipeline depth, read/write ratio, and command class, complemented by hardware-counter and symbol-level profiling. On pipelined small-payload workloads, THredis delivers 1.60–1.98× the throughput of stable Redis at 8 and 10 server threads with comparable p99 latency at the saturation point. The advantage is conditional: it fades at pipeline depth 1, on synchronous YCSB workloads, and on bandwidth-bound payloads. The profile data confirms that the hot path matches the design and that the consolidated drain

removes the per-request kernel cost that dominates both Redis and DragonflyDB on small reads.

Keywords

Redis, in-memory key-value store, execution-level pipelining, out-of-order execution, multicore scalability, lock-free synchronization, SPSC queue, hash partitioning, Tomasulo’s algorithm, pipelined workloads, NoSQL benchmarking

ACM Reference Format:

Le-Gao Chen, Cunwei Chen, Harsh Rajesh Shah, Kaiwen Zheng, and Shahin Nazarian. 2018. THredis: Tomasulo-Style Reservation Stations for Multi-Threaded Key-Value Stores. In *Proceedings of Make sure to enter the correct conference title (Conference acronym ’XX)*. ACM, New York, NY, USA, 28 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Most modern web services depend on a class of systems that sits, for the most part invisibly, between the application and the persistent database: the in-memory key-value store. When a social media feed loads in 200 milliseconds, when a logged-in session is recognized across page navigations, when a leaderboard updates as the user watches, an in-memory key-value store is typically what made the underlying operations fast enough to feel instantaneous. Redis, the most widely deployed system in this class [26, 29], has reportedly served on the order of 30 billion operations per day at Twitter on a single deployment [38] and is on the request path of every page load at Stack Overflow [33]. Beyond these public examples, Redis or a Redis-compatible system has been reported in production use across distributed MQTT brokers [9], healthcare geospatial search [22], clustered web-session management [18], railway sensor monitoring [6], IoT time-series alerting [12], financial hot/cold data tiering [17], vehicle location tracking [35], and distributed web crawling [15], among many others. More recently, the same low-latency access pattern has also become important in AI and large-language-model applications, where Redis-like systems are used for semantic caching, vector search, agent memory, and fast retrieval of repeated or similar prompts [24, 25].

This deployment breadth puts Redis’s per-server performance directly on the latency-critical path for billions of user-visible requests per day. A 100-microsecond improvement on a Redis GET multiplied across a Twitter-scale request volume is not merely a microbenchmark abstraction; it is engineering effort that the rest

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference acronym ’XX, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2018/06

<https://doi.org/XXXXXXX.XXXXXXX>

of the stack either does or does not have to compensate for. As such, the question of how fast a single Redis-compatible server can usefully be on modern hardware is a question about how much of the overall web infrastructure has to scale by adding machines rather than by better using the machines it already has.

Yet Redis was designed in an era when single-core performance was the dominant lever, and its core architecture reflects that era: command execution runs on a single thread, regardless of how many cores the underlying server has [30]. Modern servers routinely ship with 16, 32, or 64 physical cores; Redis itself can usefully employ approximately one for command execution. As core counts continue to grow and per-core frequency growth has slowed, the gap between hardware capability and Redis's use of that hardware continues to widen. Redis 6 introduced auxiliary IO threads that help with network read and write [27], but command execution remains on the single main thread.

Two production Redis-compatible alternatives have addressed this gap differently. KeyDB [32] is a Redis fork that runs multiple server threads against a shared key space protected by per-slot spinlocks, preserving Redis's broad command surface at the cost of exposing lock contention on every key space access. DragonflyDB [10, 11] is a clean-room re-implementation that partitions the key space across threads and uses a fiber-based scheduler for cross-shard coordination, avoiding most lock contention but at the cost of a more complex execution model than Redis itself. Both designs improve over Redis on multi-core throughput in some workloads, but as the cycle profiles in Section 7 show, each pays its own architectural cost: KeyDB spends 60–85% of CPU cycles on lock acquisition rather than command work, and DragonflyDB pays an unusually high per-completion kernel cost on small reads.

Neither alternative exploits a property that has been latent in Redis's protocol for years. Pipelined clients send multiple commands on the same connection without waiting for each reply – a feature the Redis protocol has supported since the beginning, mainly to reduce the cost of waiting for network round trips [29]. In principle, a server is free to execute the commands in such a pipeline batch in parallel, since they are already in flight on the wire. In practice, no production Redis-compatible server does so: they treat pipelining as a wire-level optimization that lets them read multiple commands from a single `recv` syscall, but still execute those commands one at a time on a single thread or, in the case of KeyDB and DragonflyDB, on whatever thread happened to receive the connection. The opportunity that pipelining creates *within* a single client's command stream is left on the table.

1.1 Contributions

This paper presents THredis, a multi-threaded fork of stable Redis that takes execution-level pipelining as its starting design assumption: multiple commands from the same client connection may be in flight on different worker threads simultaneously, and the server must preserve Redis's client-visible reply ordering and same-key semantics on top of that parallel execution. THredis splits server work across two specialized thread pools: IO threads that own client connections and parse incoming requests, and worker threads that own disjoint shards of the key space and execute commands in parallel.

Three mechanisms make this architecture viable for Redis-compatible workloads. The *fake-client abstraction* lets Redis's existing `void(*proc)(client*)` command-handler API run on a worker thread without modification, so the design inherits Redis's command surface unchanged. The *single-writer-per-key partition* eliminates key space data hazards by ownership rather than by locking, removing the lock-contention cost that dominates KeyDB's profile. The *consolidated reply drain* batches replies for an entire pipeline batch into a single `sendmsg` on the IO thread, so the server pays the syscall cost once for many replies instead of once per reply.

Our contributions are:

- A multi-threaded Redis-compatible architecture that supports *execution-level pipelining* – multiple in-flight commands from the same client connection executing in parallel – while preserving Redis's client-visible reply ordering and same-key semantics (Section 4).
- The fake-client abstraction, which preserves the Redis command-handler API exactly while making each in-flight request a movable execution context that can be dispatched across threads (Section 4).
- A lock-free dispatch and completion path: hash-partitioned single-producer/single-consumer queues for IO-to-worker dispatch, and an atomic per-client reply-ready bitmask for worker-to-IO completion signalling (Section 4).
- A consolidated reply drain that emits an entire pipeline batch's replies in a single `sendmsg` per drain pass, reducing the GET kernel-time fraction from 55.3% on stable Redis to 27.6% on THredis at the same operating point (Section 4, Section 7).
- A from-first-principles cost analysis of the resulting hot path (Section 5) and an empirical evaluation across `memtier_benchmark`, `redis-benchmark`, and YCSB (Section 7) showing 1.60–1.98× throughput improvement over stable Redis on pipelined balanced workloads, with cross-architecture cycle attribution explaining where each compared system spends its CPU.
- An honest accounting of where the design loses: synchronous clients, YCSB workloads A–C, and pipeline depth 1, where the per-request dispatch hop is paid without a batch to amortize it across (Section 7, Section 8).

1.2 Roadmap

The rest of the paper is organized as follows. Section 2 surveys the Redis architecture, the two production multi-threaded alternatives, KeyDB and DragonflyDB, and the broader concurrent key-value store research context. Section 3 states the client-visible correctness properties any Redis-compatible parallelization must preserve, and distinguishes them from the looser ordering allowances that make parallel execution possible at all. Section 4 presents the THredis architecture and algorithm, walking through the path of a command from receipt on the IO thread to retirement back to the socket. Section 5 analyzes the design's cost structure from first principles, including asymptotic complexity, load balance, communication and synchronization cost, and the generality–speed tradeoff created by the single-key fast path. Section 7 reports the experimental

evaluation, including the cross-architecture cycle attribution that motivates much of the discussion. Section 8 interprets the results, identifies the design’s limitations, and outlines the future work the limitations suggest. Section 9 concludes.

2 Background

This section describes the Redis-compatible key–value store landscape in which THredis operates. We first summarize Redis and two production Redis-compatible multicore systems, KeyDB and DragonflyDB, with attention to client ownership, event processing, command execution, and synchronization. We then review prior work on Redis applications and Redis-oriented optimizations, followed by broader in-memory key–value store research. Finally, we distinguish wire-level pipelining from the execution-level pipelining targeted by THredis.

2.1 Redis

Redis is an open-source in-memory key–value store widely used as a database server, cache, message broker, and streaming engine [29]. Unlike disk-oriented relational database systems, Redis serves its active working set primarily from main memory, which enables low-latency access and high throughput [29]. Although Redis exposes a key–value interface, values are not limited to byte strings: Redis provides built-in data structures including strings, hashes, lists, sets, sorted sets, bitmaps, streams, and related command families [28].

2.1.1 Applications. Redis is commonly deployed in latency-sensitive services where fast access to shared state is on the request path. Public engineering reports and case studies describe Redis being used for large-scale social-media update paths and for web application state such as caching, counters, and session-related data [26, 33, 38]. Academic and industry-oriented studies further show Redis serving as a coordination and data-management layer in distributed MQTT brokers, healthcare geospatial search, clustered web-session management, railway sensor monitoring, IoT time-series alerting, financial hot/cold data tiering, vehicle location tracking, and distributed web crawling [6, 9, 12, 15, 17, 18, 22, 35].

These workloads share two requirements that motivate continued Redis optimization: high throughput under many concurrent clients and low latency under bursty request streams. As server core counts increase, the central question is no longer only whether Redis is fast on one core, but whether Redis-compatible semantics can be preserved while exposing more command execution parallelism.

2.1.2 Redis Architecture. Redis follows a primarily single-threaded design for command execution, which avoids the complexity of concurrent access to shared keyspace data structures [30]. A main event loop uses an I/O multiplexer such as `epoll` to monitor many client connections efficiently [30]. When a connection becomes readable, Redis reads the request, parses the command, executes the command against the database, and later writes the response back to the client. Since command execution is serialized through one main thread, Redis avoids locks on the database hot path and keeps command-handler code simple (Figure 1a).

Redis also supports I/O threads, where helper threads assist the main thread with network read and write processing [27].

These threads do not change the command execution model. The main thread still owns event multiplexing and database command execution, while I/O threads only offload portions of the socket read/write path [27]. Redis I/O threads can therefore reduce network-processing overhead, but they do not allow multiple commands to execute in parallel against the database.

2.2 KeyDB

KeyDB is a Redis fork designed to improve multicore scalability through multi-threaded command execution [32]. Instead of relying on one main execution thread, KeyDB runs multiple server threads, each with its own event loop and subset of client connections [32]. This allows different clients to be parsed and executed in parallel (Figure 1b).

The main tradeoff is that KeyDB retains a shared logical dataset across threads. Because multiple server threads may access the same keyspace concurrently, KeyDB must use synchronization to preserve Redis-compatible semantics. Commands that conflict on the same data therefore incur locking or coordination overhead, and write-heavy or hot-key workloads can expose this serialization cost. KeyDB improves parallelism across independent clients, but it does not eliminate runtime conflict resolution on shared data.

2.3 DragonflyDB

DragonflyDB is a Redis-compatible database designed around a multi-threaded architecture with sharded ownership [10, 11]. Each thread runs its own event-processing path, manages a subset of client work, and owns part of the dataset [10, 11]. This design reduces lock contention because many single-key operations can execute on the thread that owns the relevant shard (Figure 1c).

The main tradeoff is cross-shard coordination. When a command or transaction touches data owned by multiple shards, DragonflyDB must coordinate execution across threads. This provides a more general execution model than a strictly single-shard design, but the additional coordination can increase latency for workloads that frequently access data across shard boundaries [10, 11].

2.4 Redis-Oriented Optimizations

Prior Redis-specific work shows that Redis performance can be limited by overheads outside the obvious command logic. FlexPoll optimizes Redis’s use of Linux `epoll` by reducing unnecessary `epoll_ctl()` calls under high-concurrency network workloads [39]. The key observation is that precisely updating read/write interest after every state transition can add system-call overhead that is significant for lightweight Redis requests. By tolerating temporary over-subscription to write-ready events and adapting when to update the kernel interest set, FlexPoll improves throughput while preserving the same request-processing semantics [39].

Redis++ targets two different Redis bottlenecks: memory fragmentation and hash-index locality [40]. Its segmented memory manager stores objects in fixed-size segments and reclaims invalidated space using a cost-benefit policy, reducing allocation fragmentation. Its two-level hash index uses a cache-line-sized bucket layout, compact tags, and SIMD-assisted tag comparison to reduce pointer chasing during lookup [40]. Redis++ is especially relevant to THredis because it demonstrates that Redis’s core hash path is a

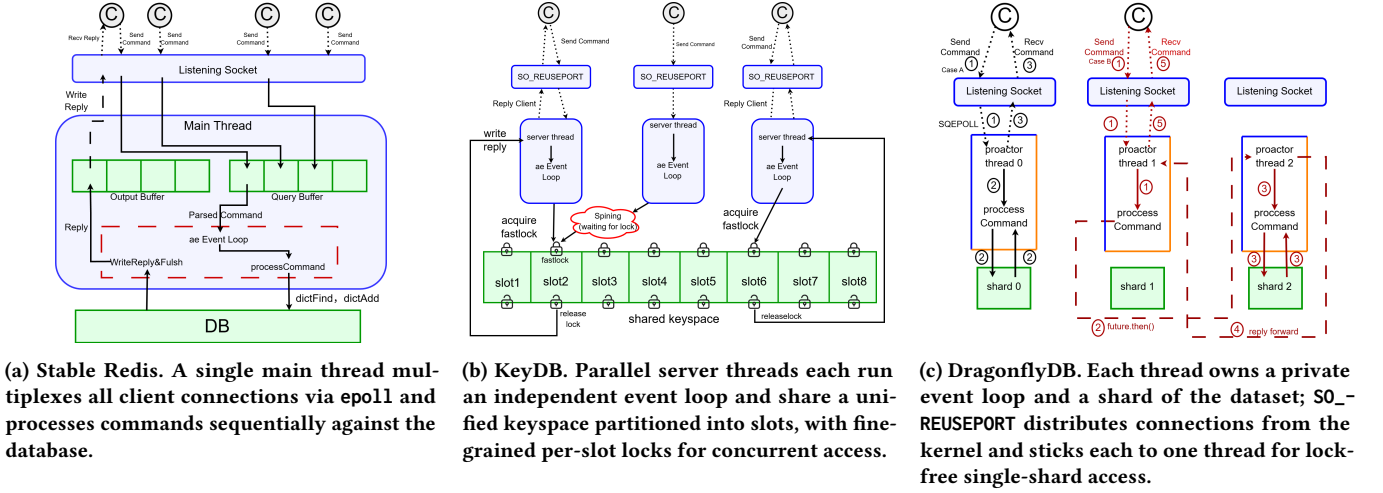


Figure 1: Threading and execution models of the three baseline Redis-compatible systems compared in this paper.

meaningful optimization target, and that using a faster hash such as `xxHash` can be beneficial in a Redis-derived system when the hash is paired with an index design that manages collision behavior explicitly [40].

Recent work by Chen et al. explores another Redis bottleneck: persistence. Their `io_uring`-based Redis snapshotting path replaces synchronous persistence operations with asynchronous I/O and shows significant gains for larger persistence workloads [5]. More generally, Jasny et al. argue that modern asynchronous I/O interfaces such as `io_uring` only deliver substantial database-system benefits when they are integrated with architectural changes such as batching and asynchronous execution [13]. THredis is complementary to these I/O-focused efforts: instead of optimizing persistence or storage I/O, it targets execution-side serialization in Redis-compatible command processing.

2.5 Concurrent Key–Value Store Research

A broader line of systems research studies how in-memory key–value stores can exploit parallel hardware while controlling synchronization overhead.

FASTER introduces an epoch-based synchronization protocol that allows threads to coordinate access to shared state without a single global lock, achieving very high single-machine throughput for concurrent key–value operations [4]. MICA takes a more partitioned approach: it combines Intel DPDK with NIC-level flow steering so requests are routed to the core that owns the relevant data partition, avoiding cross-thread synchronization on the hot path [19]. Masstree focuses on cache-efficient ordered indexing and read-heavy workloads, using optimistic concurrency control and version validation so lookups can avoid writing to shared memory [21].

Parallel transactional execution raises a related but stronger correctness problem. Silo uses optimistic concurrency control with per-thread epochs rather than a centralized timestamp, allowing transactions to validate speculatively while scaling on multicore hardware [37]. Although Redis transactions differ from general

OLTP transactions, Silo is a useful reference point because it shows how commit ordering and validation can be separated from speculative execution.

Network-path research further shows that high-throughput key–value stores are often limited by request transport and synchronization, not only by command computation. HERD demonstrates that carefully structured two-sided RDMA communication can outperform one-sided RDMA reads for key–value workloads by combining direct request placement with efficient reply delivery [14]. These systems differ substantially from Redis compatibility goals, but they motivate a common design principle: parallelism is most effective when request routing, ownership, and synchronization are aligned.

2.6 Benchmarking and Workload Characterization

Evaluation of key–value stores commonly uses the Yahoo! Cloud Serving Benchmark (YCSB), which defines standard read/write mixes and provides a framework for comparing cloud serving systems [8]. Prior workload analysis of Facebook’s Memcached deployment shows that production key–value workloads often have skewed access distributions, small objects, and read-heavy mixes, making locality and synchronization behavior important to performance [2]. Recent comparative studies continue to use YCSB-style workloads to evaluate Redis and other NoSQL systems such as Aerospike, DragonflyDB, MongoDB, Cassandra, Memcached, MySQL, and PostgreSQL [1, 23, 31].

These benchmarks are useful because they expose different pressure points: read-heavy workloads stress lookup and reply generation; write-heavy workloads stress update serialization; larger payloads stress memory movement and socket output; and high client counts stress event processing and scheduling. THredis uses this context to evaluate whether execution-level pipelining improves performance across both microbenchmark and mixed-workload settings.

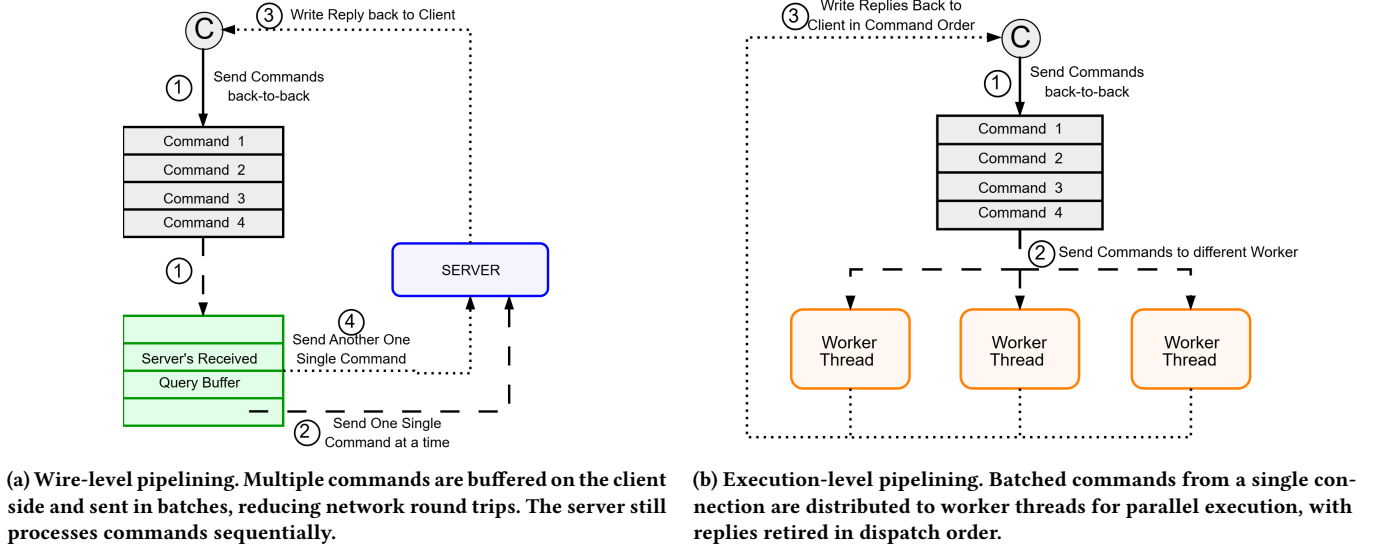


Figure 2: Wire-level vs. execution-level pipelining. Production Redis-compatible servers support (a); THredis adds (b).

2.7 Wire-Level vs Execution-Level Pipelining

2.7.1 Wire-Level Pipelining. Wire-level pipelining is the standard Redis pipelining model. A client sends multiple commands back-to-back without waiting for each reply, allowing the server to read a batch from the socket and allowing the client to amortize network round-trip time. The server, however, may still process the commands one at a time. The speedup therefore comes primarily from reducing client-visible network idle time, not from making the server execute multiple commands from the same connection concurrently (Figure 2a).

This distinction matters because production Redis-compatible servers, including multi-threaded systems such as KeyDB and DragonflyDB, support client pipelining as a protocol-level optimization. They can reduce round-trip overhead and may parallelize work across different clients or shards, but pipelining itself does not necessarily mean that one connection’s commands are simultaneously in flight through independent execution units.

2.7.2 Execution-Level Pipelining. Execution-level pipelining, in contrast, allows multiple commands from the same connection to be in flight inside the server at the same time. Commands may execute on different worker threads and complete out of order, while the server must still preserve Redis-compatible reply order and same-key semantics at the socket boundary. The speedup comes from genuine parallel execution of a single connection’s command stream, not merely from hiding round-trip latency (Figure 2b).

This model has analogues in other systems. Out-of-order processors use mechanisms such as Tomasulo’s algorithm to issue independent instructions to functional units while preserving correct retirement order [36]. Network protocols such as HTTP/2 also separate connection-level transport from concurrently processed logical streams [34]. Redis-compatible key-value stores, however, have largely treated pipelining as a wire-level batching mechanism rather than a server-side execution model. THredis targets this gap

by adding execution-level pipelining while preserving client-visible Redis semantics.

3 Requirements

Before describing our server architecture, we must articulate the client-visible correctness properties that must be preserved. If our implementation fails to satisfy these requirements, it is considered invalid regardless of any performance improvements. Redis’s available commands and execution path expose guarantees that every Redis client or application relies on [29, 30]. We have divided those properties into two specific categories: reply ordering and keyspace data hazards. Our architecture design aims to satisfy these requirements while allowing genuine parallelism across commands that address different keys.

3.1 Reply Ordering

Non-pipelined clients issue one command and wait for the reply before issuing the next command. Ordering in this case is trivial, as there is only ever one command in flight on the connection at any given time, so the server is free to process it in any internal order with no observable effect on the client’s side.

3.1.1 Pipelined Reply Ordering. Pipelined clients can send multiple commands before waiting for any replies and expect replies to be returned in the same order as the commands that produced them [29]. For example, if a client sends `SET k v` followed by `GET k`, the client expects `+OK` followed by the value `v` in that order. If a reply arrives out of order, it does not only degrade correctness but also causes clients to associate the wrong reply with the wrong request, potentially corrupting the rest of the connection by misaligning the response stream.

3.1.2 Causal Ordering. Causal ordering refers to the requirement that when two commands issued by the same client within a pipeline access the same key, the later command must observe the

effects of the earlier one, reflecting the underlying data dependency between them [29]. For example, a client that sends **SET k v1**, **SET k v2**, followed by **GET k** must receive **+OK**, **+OK**, and then **v2**; it cannot receive **v1** or **nil**. A parallel design that fires those commands independently with no synchronization can produce stale replies, and from the client's perspective this is indistinguishable from the database losing their write.

Crucially, this constraint only applies to commands that address the same key. Commands in the same pipeline that address different keys have no data dependency and thus do not need to execute in issue order. For example, if a client sends **SET k1 v1**, **SET k2 v2**, **GET k1**, any execution scheduling that results in **SET k1 v1** being committed before **GET k1**, and that emits replies in issue order, is indistinguishable to the client. In this case, the server is free to execute **SET k2 v2** before, during, or even after either of the other two commands, because **k1** and **k2** have no data dependency. Thus parallel execution across different keys is not only permitted but is also deeply ingrained in our design; this follows the broader direction of multicore in-memory database and key-value store designs that exploit independence across records or partitions [4, 19, 37].

This matches the classical out-of-order execution discipline in CPU pipelines, which addresses similar hazards. Registers in CPUs, like keys in our server, must respect data-dependency hazards, though hazards on disjoint resources may be reordered freely. Retirement observable to the outside world must be in program order, or in our case, in request order. The correspondence with Tomasulo's algorithm is direct: workers can execute out of order, but replies must retire in order [36].

3.2 Keyspace Data Hazards

Since the keyspace is mutable and shared across threads, if two threads execute commands on the same key concurrently without synchronization, three hazards may occur. First is the write-after-write (WAW) hazard, where two concurrent writes on the same key race on the dictionary entry. This may not only result in lost data but may also result in memory corruption and crashes. Second is the read-after-write (RAW) hazard, where the reading thread may observe the key's value while the writing thread is mid-update. This may not only result in a wrong value being seen by the reader, but also in invalid pointers or partially allocated objects, leading to crashes. Third is ordering inversion, where due to CPU architecture or compiler reordering of memory accesses, one thread's read may observe a later write from another thread before it observes the earlier write that was supposed to have preceded it.

A parallel execution design that ignores these hazards is not only slower than correct, it is also incorrect. These hazards may manifest as data corruption, memory-safety violations, non-deterministic replies, and even server instability and crashes.

4 Design

This section describes the THredis architecture and algorithm. We organize this section around the path a command takes through the server from the moment a request is received. The background

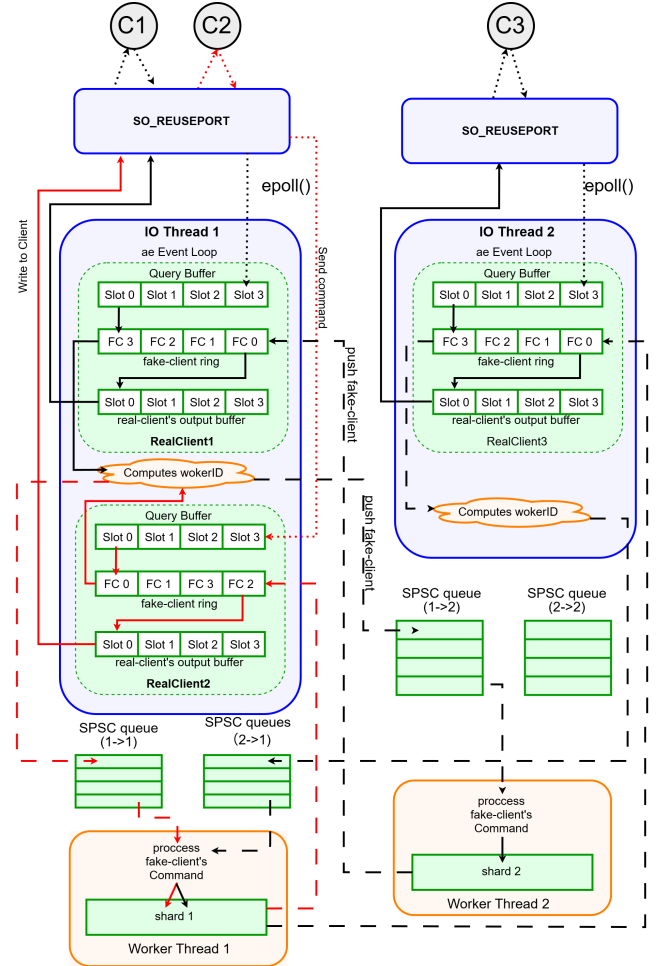


Figure 3: THredis architecture overview. Multiple IO threads accept connections via `SO_REUSEPORT`, parse requests, and dispatch commands to worker threads through lock-free SPSC queues. Worker threads execute commands on sharded keyspaces, and replies are merged and flushed by IO threads in dispatch order.

and motivation for execution-level pipelining are described in Section 2.7; this section focuses on the mechanisms THredis adds to make it possible.

4.1 Overview

THredis consists of two specialized thread types: *I* IO threads that own client connections with their own multiplexer and event loop, and *W* worker threads that execute on their own shard of the keyspace. This division departs from Redis's single command-execution thread and IO-helper model while retaining Redis-compatible command semantics [27, 30]. Figure 3 gives the high-level view of the resulting architecture; the rest of this section walks through the mechanisms that make it work.

The two types of threads communicate through two lockless mechanisms: a Single Producer Single Consumer (SPSC) queue for

command dispatch from IO threads to workers, following the single-producer/single-consumer synchronization pattern [16], and an atomic bitmask **reply_ready_mask** for signaling from the worker back to the IO thread.

4.2 Fake Client Abstraction

Before introducing the data structures that carry commands between threads, we describe a property of Redis’s internal command API that motivates a central abstraction of our design. This abstraction only works on an architecture that separates IO and execution into different threads. Without that separation, there is no parallel dispatch target, and the abstraction has nothing to hand execution state off to. The fake client abstraction does not introduce parallelism by itself; rather, it enables safe transfer of execution state between IO and worker threads once such parallelism is present.

4.2.1 Redis’s client-centric command API. Redis structures command execution around the client, not the request [30]. Every command handler has the signature `void (*proc)(client *c)`, and all of the command’s input, mutable state, and output reside on the client struct: the argument vector at `c->argv`, the target database at `c->db`, the in-progress command metadata at `c->cmd` and `c->argc`, and the reply buffers at `c->buf` and `c->reply`. The command handler calls `addReply(client, ...)` to write to the client’s output buffer and executes the command on `c->db` via dictionary operations. The server is expected to flush `c->buf` to the socket later.

This API is natural for Redis’s single-threaded execution model, described in Section 2.1, since there is at most one command executing at any moment and the client struct can serve as both the connection wrapper and execution context. It greatly complicates execution-level pipelining because a command cannot simply be sent to a worker thread when the execution context is the entire client struct. Reconstructing the command handler API to take only a request context would require rewriting several hundred command implementations across string, hash, list, sorted-set, set, bitmap, HyperLogLog, and stream commands.

4.2.2 Fake Client Mechanism. The key challenge in enabling execution-level pipelining is that command execution state is stored directly on the client object and reused across requests. The IO thread must continue parsing subsequent commands and therefore needs to reuse the client’s command execution fields, while earlier commands may still be executing and require those same fields to remain valid.

THredis resolves this conflict by introducing the fake client abstraction. Instead of executing commands directly on the real client, the IO thread transfers the execution state of each request into a fake client which owns its own command execution state. The fake client is then dispatched to the worker thread where the worker invokes the command handler directly on the fake client with `fake->cmd->proc(fake)`. Because the command handler receives a `client *` as it always has, no command implementations need to be modified.

This split—real clients own connections, fake clients own executions—is the structural premise on which the remainder of the design is built. The real client reads commands from the

wire and writes replies back; the fake clients serve as reservation stations in which commands wait for and receive their results.

4.2.3 Why existing servers do not need fake clients. As described in Sections 2.1–2.3, existing Redis-compatible servers do not allow multiple commands from the same connection to execute concurrently [10, 27, 32]. Because they treat pipelining as wire-level pipelining rather than execution-level pipelining, the client struct can be reused across requests without preserving multiple concurrent execution states. The fake client abstraction is therefore unnecessary in these designs, but becomes essential once commands from the same connection are allowed to execute concurrently.

4.3 Data Structures

There are three new data structures in THredis: a fake-client ring that stores pipelined request execution states, a worker queue that carries fake clients from IO threads to workers, and the **reply_ready_mask** that signals IO threads when replies are ready.

4.3.1 Fake-client Ring. Each client connection, hereafter the real client, owns a ring of pipeline depth P fake clients that hold the execution state of in-flight commands. This ring is indexed by two counters: *dispatchid*, which advances when a command enters the ring, and *flushid*, which advances when replies have been written to the socket. A command’s slot is $\text{dispatchid} \& (P - 1)$, assuming P is a power of two. Dispatch stalls when the ring is full, which occurs when $\text{dispatchid} - \text{flushid} = P$.

4.3.2 Worker Queue. Each worker exposes S single-producer single-consumer (SPSC) queues, one per producer (the main thread and each IO thread), where each producer exclusively owns its queue. The elements in these queues are fake clients carrying their execution state. Producers push a fake client into the queue with a single atomic release-store to the tail index, while the consumer batch-pops by advancing the head with a single release-store, reading up to 32 slots at a time. Because each queue has exactly one producer and one consumer, no queue mutex is required.

4.3.3 reply_ready_mask. Each real client owns a 32-bit atomic word which we call the **reply_ready_mask**. Bit n signals that slot n ’s reply is ready for flushing. Workers set these bits with `atomic_fetch_or`, then IO threads read them with `atomic_load_acquire`. After the IO thread flushes the reply, it sets the bit back to 0 with `atomic_fetch_and`. This way multiple workers can write to the same real client’s **reply_ready_mask** at the same time with no contention.

4.4 Command Dispatch: IO Thread

4.4.1 Client Connection and Event Multiplexing. Both IO threads and the main thread accept connections via **SO_REUSEPORT**, allowing multiple sockets to bind to the same port simultaneously. The kernel hashes the incoming 5-tuple and enqueues the connection to exactly one listening socket. Each thread’s `accept()` dequeues from its own socket queue with no synchronization with other threads. As a result, client-to-thread load balancing is handled by the kernel, avoiding a userspace accept dispatcher, shared accept queue, work-stealing, or rebalancing logic. Each connection

is also sticky to its assigned IO thread for the connection's lifetime, preserving cache locality.

Each IO thread then runs its independent ae event loop with its own epoll instance, monitoring only its private listening sockets and connections [20, 39]. There is no shared event loop, epoll descriptor, or synchronization between IO threads. Any incoming request on connection C is observed exclusively by the IO thread that owns C and is handled entirely on that thread. With I IO threads, the server maintains I independent epoll instances and event loops.

This connection model follows the same high-level client partitioning used by KeyDB and the proactor-style model used by DragonflyDB, described in Sections 2.2 and 2.3 [10, 32]. THredis adopts **SO_REUSEPORT** because the alternatives—a single accept dispatcher or multiple threads contending for one shared socket—introduce contention and user-space coordination.

4.4.2 Dispatch to Workers. When an IO thread receives a request, it parses the request into the client's data structures, populating the **argv** and **cmd** fields. Up to this point, the execution path is identical to stable Redis.

Dispatch diverges at command execution. Rather than executing the command as in stable Redis, the IO thread moves the elements required for execution from the real client into the fake client at *dispatchid* & $(P - 1)$ via pointer assignments: *fake*->*argv* = *real*->*argv*, *fake*->*cmd* = *real*->*cmd*. The corresponding fields on the real client are then cleared by assigning them to NULL pointers. The *argv*, the *robject* objects it points to, and their SDS payloads are not copied; only references are transferred. After this move, the fake client owns the execution state and the real client is free to parse subsequent pipelined commands. Moving pointers instead of copying execution state keeps dispatch cost constant: a 4-byte SET and a 4-megabyte SET both move the same handful of pointers (Figure 4).

The IO thread then selects a worker:

$$w = \text{xxhash64}(\text{key}) \& (W - 1),$$

where W is the worker count and must be a power of two. We use xxHash [7] because it is a fast non-cryptographic hash suitable for inexpensive load distribution, while Redis's dictionary path uses SipHash-style hashing for stronger short-input and hash-table-collision resistance [3]. Worker dispatch is not exposed to adversarial hash-table probing, so key collisions affect load balancing but not correctness. This is a narrower use of xxHash than Redis++, which uses xxHash as part of a high-performance Redis-compatible two-level hash-index design [40]. THredis uses xxHash only as a dispatch hash: it decides which worker owns a command, but it does not replace Redis's dictionary hash during command execution. Later during command execution, this dispatch hash is not reused. The power-of-two mask also lets the compiler replace integer division with a single AND instruction in the hot path.

The fake client is then pushed to the selected worker's SPSC queue tail slot with a single atomic release-store, and *dispatchid* is incremented. Dispatch returns to the event loop without waiting for completion. Subsequent commands on the same connection can be read and dispatched freely until the fake-client ring is filled.

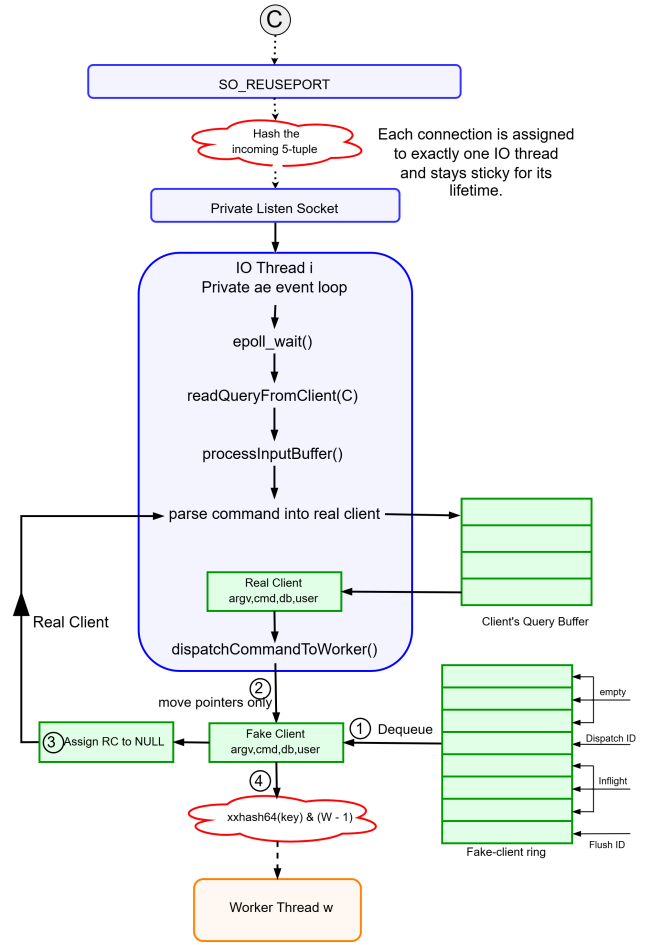


Figure 4: Command dispatch from IO threads to workers. IO threads parse client requests into real clients, transfer execution state into fake clients, and dispatch them to worker threads via SPSC queues. The fake client's slot in the per-client ring carries the sequence number used for in-order retirement.

4.4.3 Design Alternative: Redis IO Threads. Redis IO threads, described in Section 2.1, do not participate in the same dispatch path [27]. Redis keeps a single ae event loop with a single epoll instance on the main thread regardless of IO thread count. Its IO threads only assist with receiving commands and writing replies; command execution is still handed back to the single main thread. This preserves Redis's serial execution path and avoids synchronization overhead for database commands, but it also means Redis IO threads do not provide execution-level pipelining.

4.5 Execution: Worker Threads

Each worker thread runs an infinite loop over its I SPSC queues, where I is the IO thread count plus the main thread, popping up to 32 fake clients per queue per round. The popped batch is processed in three phases: hash, prefetch, and execute (Figure 5).

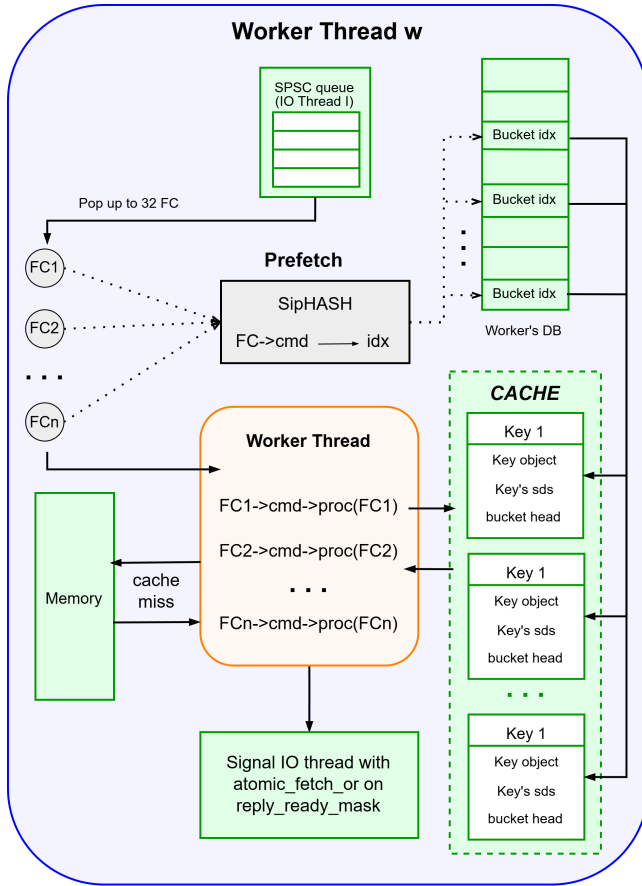


Figure 5: Worker thread execution pipeline. Each worker processes batches of fake clients through three stages: key hashing (SipHash), software prefetching of dependent cache lines, and command execution. The prefetch phase warms the cache lines that the execute phase of later commands in the batch will depend on.

4.5.1 Hash. The first thing the worker does on each batch is the work that would otherwise be the first thing the command handler does: it hashes each command’s key against the shard’s dictionary using Redis’s standard SipHash-1-2 to compute the destination bucket index. This index enables prefetching in the next phase; without it, the worker would know which shard to access but not which cache line to warm.

This second hash serves a different purpose from the dispatch hash. The dispatch hash in Section 4.4.2 uses xxHash64 to choose the worker shard. The worker uses SipHash to find the exact bucket inside that shard. This slower, hardened hash is used for the actual hash table lookup so the shard inherits Redis’s hash-DoS protection and key-collision handling.

4.5.2 Prefetch. With the bucket index from the hashing phase, the worker issues software prefetches in stride order across the batch for three lines per command: the key object, the key SDS, and the bucket head. The bucket-head prefetch is used later by

dictFind or dictAdd, where the cache miss would otherwise be paid synchronously during execution.

The prefetches are issued across the batch so the dependent loads of the first command’s execution overlap with the prefetches of later commands. This also lowers the visible cache-coherence latency of the batch’s fake-client migration.

Software prefetching is not novel to THredis; Stable Redis, KeyDB, and DragonflyDB also use prefetching. Our contribution is its placement. Since THredis separates client IO from execution, the correct shard-local cache lines are only known on the worker side, so prefetching moves from the IO thread into the worker through **workerPrefetchBatch**. This lets the worker warm the cache lines for commands it is about to execute on its own shard.

4.5.3 Execution. The worker then iterates the batch, calling the command’s handler directly on each fake client (`fake->cmd->proc(fake)`).

The execution phase begins by hashing the key again to probe the actual location of the data. Although the request has already been hashed twice—once for worker selection and once during the prefetch stage—the result is not carried forward into execution, and the handler recomputes the hash as part of the dictionary lookup path. As a result, the system pays the SipHash-1-2 cost twice per command. This is not unique to our design; stable Redis incurs the same double-hash cost in its own prefetch path.

This handler reads from and writes to the worker’s private database shard. No locks are required here because the shard is accessed by exactly one worker thread. The handler then produces the reply into the fake client’s static buffer and, if needed, its own overflow reply list.

4.5.4 Signal. After the handler returns, the worker atomically sets the bit corresponding to `fake_slot` in the real client’s `reply_ready_mask` using `atomic_fetch_or_release`.

4.5.5 Yield. If the worker finds all SPSC queues empty, it fires a short burst of PAUSE instructions, roughly 500 ns each, before falling back to `sched_yield`. This avoids scheduler context switching during the short gaps between bursts of dispatch, characteristic of pipelined workloads.

4.5.6 Design Alternative: KeyDB Per-slot Locking. KeyDB’s solution to the keyspace hazard problem is through a custom ticket spinlock, or fastlock, that guards each slot of the keyspace. Server threads can read and parse in parallel, but every write on a given slot is serialized through the slot’s exclusive lock. Under uniform-random key distributions at saturating loads, fastlock can avoid kernel transitions but does not avoid serialization.

4.6 Retirement: Reply Drain

The IO thread’s event loop invokes `handleWorkerReplies` each iteration. For each real client with any in-flight work (`dispatchid ≠ flushid`), the drain snapshots `reply_ready_mask` with `acquire_load`, then walks the ring from `flushid` upwards. If the bit for slot `flushid & (P-1)` is not set to 1, the drain stops there. Later slots may be ready, but in-order retirement requires not advancing past an in-progress slot. If the slot is ready to flush, we call `addReplyFromClient(real, fake)`, which `memcpy`s the fake client’s static reply

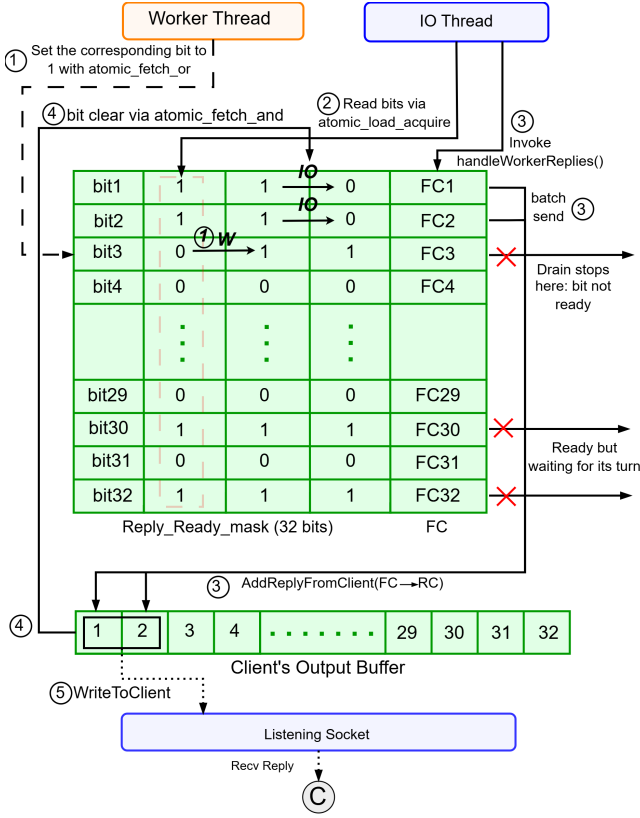


Figure 6: In-order reply handling. Worker threads signal completion by setting bits in a per-client `reply_ready_mask`. The IO thread walks the fake-client ring in `flushid` order, splicing replies into the real client's output buffer until it hits a slot that is not yet ready. A single `sendmsg` per drain pass emits the assembled batch, preserving dispatch order despite out-of-order worker completion.

buffer into the real client's output buffer or `listJoins` the fake client's overflow reply list into the real client's list. The `reply_ready_mask` bit for that specific slot is then reset with `atomic_fetch_and`, the fake client's command state is retired via `commandProcessed`, and `flushid` advances (Figure 6).

After the `handleWorkerReplies` loop, if any fake client was spliced, the IO thread issues a single `writeToClient` call on the real client, which drains all its accumulated reply bytes in one `sendmsg` system call.

Because the drain walks in `flushid` order and `memcpy`s bytes into the real client's buffer sequentially, the bytes in the buffer at the time of the system call are already in dispatch order. Because `writeToClient` only fires once per drain pass rather than once per request, the system call cost is minimized per pipelined batch: at pipeline depth P , the system call count is $1/P$ of the cost a per-request design would incur.

We also considered and rejected the idea of flushing fake clients directly. A per-fake-client `writeToClient` eliminates the `memcpy`

into the real client's buffers but requires significantly more system calls under pipelined workloads.

4.7 Correspondence to Tomasulo's Algorithm

Tomasulo	THredis
Fetch stage	epoll + query buffer
Issue stage	IO-thread dispatch
Reservation station	Fake client + SPSC queue
Tag / renamed register	Key hash → worker assignment
Register renaming	Hash partitioning (structural single-writer)
Functional unit	Worker thread
Write-back	Fake client reply buffer write
Common Data Bus	Completion signal (<code>reply_ready_mask</code>)
Reorder Buffer	Fake-client ring with <code>dispatchid</code> & <code>flushid</code>
Commit	In-order drain

Table 1: Correspondence between components of Tomasulo's algorithm and THredis. The most significant divergence is hazard resolution: Tomasulo renames registers to eliminate write hazards dynamically, while THredis eliminates them structurally by assigning each key to exactly one writer thread.

The dispatch, execution, and retirement stages described above form the structure of Tomasulo's algorithm for out-of-order instruction execution in hardware processors. Table 1 summarizes the mapping.

The properties preserved from Tomasulo are out-of-order execution, since workers may complete in any order; in-order retirement, since the drain refuses to advance `flushid` past the oldest in-progress slot; and dataflow hazard resolution, since keys with the same tag are serialized at the functional unit while keys with different tags execute concurrently.

The principal divergence from Tomasulo is the hazard-elimination mechanism. Tomasulo renames registers so that multiple in-flight instructions writing the same architectural register map to disjoint physical registers; THredis instead assigns each key to a single writer thread at the granularity of the hash partition. We trade the dynamic flexibility of renaming for the static simplicity of a compile-time partition, which is appropriate for a workload in which the "register file" (the keyspace) is orders of magnitude larger than the number of functional units.

This correspondence is useful for intuition but is not the framing of our contribution. The novelty in THredis lies in the lock-free SPSC handoff, the bitmask-encoded completion signal, the consolidated drain splice, and the structural single-writer invariant—not in the Tomasulo analogy itself.

4.8 Correctness Properties

THredis preserves the client-visible semantics of Redis while enabling execution-level pipelining. Correctness follows from three key design properties.

Single-writer per key. Each key is assigned to exactly one worker thread at dispatch time. Commands operating on the same key are therefore serialized in the worker's input queue, eliminating write-write and read-write conflicts without requiring locks.

In-order reply emission. Each client maintains a fake-client ring indexed by *dispatchid* and *flushid*. The IO thread drains replies strictly in *flushid* order and does not advance past incomplete slots. As a result, replies are emitted to the socket in the same order as the corresponding requests, regardless of execution completion order.

Cross-thread visibility. Workers signal completion using *reply_ready_mask* with release semantics, while the IO thread observes readiness using *acquire_loads*. This guarantees that all updates performed by a worker—including keyspace mutations and reply writes—are visible before the corresponding reply is emitted.

Together, these properties ensure correct serialization of same-key operations, preservation of client-visible ordering, and absence of data races in the execution path.

5 Performance Properties

This section characterizes THredis’s cost structure from first principles. The goal is not to present measured results, but to explain where the architecture spends work on the hot path and why those costs differ from stable Redis, KeyDB, and DragonflyDB [10, 30, 32].

We analyze asymptotic cost, load balance under hash-partitioned dispatch, communication and synchronization overhead, and the generality–speed tradeoff created by THredis’s narrower single-key fast path. The mechanisms being analyzed—fake-client rings, SPSC queues, reply masks, dispatch, worker execution, and retirement—are introduced in Sections 4.3–4.6.

5.1 Complexity Analysis

Each operation along the request path is constant-time in the number of in-flight requests, with one bounded-by-ring-depth exception during reply drain. Table 2 summarizes the hot-path cost.

Table 2: Per-operation cost of the THredis hot path. P is the fake-client ring depth, B is the worker batch size, k is the number of ready slots drained in one pass, and s is the reply size in bytes.

Operation	Cost	Implementation
Worker selection	$O(1)$	XXH64 + power-of-two AND
Slot allocation	$O(1)$	<i>dispatchid</i> & $(P-1)$
SPSC enqueue	$O(1)$	pointer write + release-store
SPSC batch dequeue	$O(B)$	<i>acquire-load</i> + B pointer reads
Reply bit set	$O(1)$	atomic <i>fetch_or</i>
Reply bit check	$O(1)$	<i>acquire-load</i> + bit test
Reply drain	$O(k)$	sequential walk from <i>flushid</i>
Reply splice	$O(s)$	<i>memcpy</i> + <i>listJoin</i>
Per-request total	$O(1)$	bounded atomics and pointer movement

The drain’s $O(k)$ cost is bounded by the fake-client ring depth P because the IO thread stops at the first non-ready slot to preserve in-order retirement. In steady state, the drain usually processes only a small number of slots per pass, so the amortized per-request cost remains constant.

The reply splice’s $O(s)$ cost applies only to bytes copied from the fake client’s short reply buffer. Replies that overflow into the linked-list reply chain are spliced in $O(1)$ via *listJoin* and are not copied during the splice. For common Redis replies such as +OK,

integers, and short strings, the splice is therefore a short bounded *memcpy*.

Scaling with database size and request rate. All per-operation costs in Table 2 are $O(1)$ in database size N . For keyspaces of $N = 2 \times 10^6$ keys and payloads up to 1 KiB, the total working set is at most 2 GiB: comfortably in DRAM but larger than typical L3 cache capacity. This places the compared systems in the same memory-hierarchy regime. Throughput scales with worker count W until IO-thread parsing, network bandwidth, or another shared resource becomes the bottleneck rather than worker compute.

5.2 Scaling limits

Although THredis is not a single fixed computation with a simple Amdahl-style serial fraction, its scaling is still bounded by the non-parallel portions of the request path. The IO thread must parse requests, preserve per-connection dispatch order, and retire replies in order. The socket output path and memory bandwidth also become shared limits as payload size increases. In addition, the single-writer-per-key invariant intentionally serializes commands that target the same key on one worker. These limits explain why THredis gains the most on pipelined, small-payload, multi-key workloads, but gains less at pipeline depth 1, on synchronous YCSB clients, on bandwidth-bound large values, and on workloads whose parallelism is limited by a small number of hot keys.

5.3 Load Balancing

Worker selection uses the command key hash reduced by a power-of-two mask. The dispatch mechanism itself is described in Section 4.4.2; the important performance property is that the power-of-two mask avoids integer division and branch-heavy dispatch logic.

Uniform key distribution. For uniformly random keys, XXH64 distributes commands evenly across workers. With N active requests and W workers, the expected load per worker is N/W , with variance shrinking as the number of requests grows. In this regime, imbalance is dominated by normal randomness in the workload and differences in command cost, not by the dispatch mechanism itself.

Skewed key distribution. The single-writer-per-key invariant from Section 4.8 is also the primary load-balancing limitation. The choice to partition by ownership rather than by lock follows prior multicore key–value store designs that align request routing with data ownership to avoid cross-thread synchronization on the hot path [19, 21]. If one key receives a disproportionate share of writes, all writes to that key must execute on the one worker that owns it. A workload where one key receives half of all writes will saturate that worker while other workers may remain underutilized.

THredis does not currently split hot keys, replicate writable keys, or migrate individual keys between workers at runtime. These mechanisms would require cross-worker coordination or would weaken the single-writer invariant that removes locks from the hot path. A natural extension is to move cold keys away from a hot worker while keeping the hot key itself single-owned, but the current design favors a simple static partition to keep dispatch cost low.

5.4 Communication and Synchronization Cost

THredis's cross-thread cost comes from the same places as its synchronization: dispatch from IO thread to worker, completion signaling from worker to IO thread, and reply transfer during drain. The detailed mechanisms are described in Sections 4.4.2 and 4.6; here we focus on their cost.

Dispatch handoff. The IO thread writes a `client*` pointer into the selected worker's SPSC queue and advances the tail with a `release-store`. The worker later observes the tail with an `acquire-load` and reads the pointer. This is both the communication mechanism and the synchronization mechanism for dispatch. The command payload is not copied; the fake client carries references to the command state, so dispatch transfers ownership of a small execution context rather than moving request bytes.

Completion handoff. After executing the command, the worker sets the corresponding bit in the real client's `reply_ready_mask`. The IO thread later reads the mask and clears the bit after retirement. This bit is both the readiness notification and the synchronization point that makes the worker's reply writes visible to the IO thread. Multiple workers may set different bits in the same word, but the IO thread is the only consumer that clears bits after draining replies.

Reply handoff. For a reply of size s , the IO thread reads the fake client's reply data and splices or copies it into the real client's output path. For replies stored in the fake client's short buffer, this costs approximately $\lceil s/64 \rceil$ cache-line transfers. For overflow replies stored in the linked-list reply chain, `listJoin` avoids copying the list itself, though the IO thread still reads the reply data when writing to the socket.

The resulting hot path has no mutex, no shared work queue, no scheduler object, and no per-key lock acquisition. Same-key synchronization is handled structurally by worker ownership: commands for the same key are sent to the same worker and execute in that worker's queue order. Cross-thread synchronization is therefore limited to the two places where ownership actually changes: the SPSC dispatch handoff and the reply-ready completion handoff.

This differs from lock-based and scheduler-based designs. KeyDB resolves key-space hazards through per-slot locking, so contention creates repeated cache-line movement on the lock itself and forces conflicting writers to wait on shared synchronization state [32]. DragonflyDB's future/fiber path supports more general cross-shard execution, but introduces scheduler-managed request and reply coordination [10]. THredis instead pays only the cache-coherence traffic needed to move one command to its owning worker and return its reply.

For common small replies, the typical cross-thread traffic is a few cache lines: one for dispatch, one for completion signaling, and one or more for the reply payload. This is still real communication cost, but it is bounded, direct, and proportional to the state that must cross from IO to worker and back. The remaining major costs are command parsing, dictionary lookup, command execution, reply formatting, socket I/O, and cache misses caused by the key-space itself.

5.5 Discussion: The Generality-Speed Tradeoff

THredis deliberately optimizes a restricted command surface. The fast path assumes that each dispatched command has one key, that the key maps to one worker, and that the command can complete without cross-worker coordination. Under these assumptions, the dispatch path reduces to a worker selection hash, one SPSC enqueue, one worker execution, one completion bit set, and one in-order drain.

This restriction is what makes the fast path inexpensive. A more general system must support commands whose keys span multiple shards, commands that require coordination between workers, and command paths that may suspend and resume execution. Those features improve coverage and composability, but they add scheduling and synchronization machinery to the path.

KeyDB and DragonflyDB represent two different points in this design space, as described in Sections 2.2 and 2.3. KeyDB keeps Redis compatibility by resolving conflicts dynamically through locks, which preserves a broad command surface but makes contention visible on the hot path [32]. DragonflyDB uses a future/fiber model that expresses cross-shard scatter-gather commands naturally, making it suitable for a wider command surface [10]. THredis takes the opposite design point: single-key commands use a direct fast path, while commands that do not fit the model cannot use the same low-cost dispatch mechanism.

The tradeoff is therefore explicit. THredis is less general than systems that support arbitrary cross-shard execution on the same path, but it is faster for the dominant single-key case because it avoids lock acquisition, fiber scheduling, and cross-worker coordination. For Redis-style workloads dominated by GET, SET, and other single-key operations, this narrower surface makes the per-command overhead small enough for execution-level pipelining to be practical.

This also makes future optimization easier. Because the fast path applies to a bounded set of command shapes, optimizations such as cached hash results, specialized reply formatting, per-worker allocation, and NUMA-aware shard placement can be implemented without changing every command handler. The restricted surface is therefore not only a current performance tradeoff, but also a way to keep future fast-path optimizations tractable.

6 Evaluation Methodology

6.1 Workloads

We use three load generators because no single benchmark captures the full behavior of a Redis-compatible server. THredis is designed primarily to improve throughput for pipelined clients by allowing commands from the same connection to execute concurrently. Therefore, benchmarks that do not stress pipelined request streams are useful but incomplete. YCSB is a standard benchmark in database systems research and is therefore required for comparability [8], but its Redis binding uses a synchronous request-response model and does not expose the main bottleneck THredis targets. Prior NoSQL and key-value benchmarking studies also show that benchmark choice can materially affect observed performance, motivating our use of multiple load generators rather than a single workload source [1, 23, 31]. `redis-benchmark` supports the broadest set of Redis data structures among the tools we use and is useful

for isolating per-command behavior, but its synthetic command streams do not model full application workloads. `memtier_benchmark` fills this gap by generating mixed read/write application-style workloads with pipelining and tail-latency reporting, although it covers a smaller subset of Redis commands and data structures.

For `redis-benchmark` and `memtier_benchmark`, the number of client threads is matched to the server thread count being evaluated. This keeps client-side parallelism proportional to the server configuration and reduces the risk that a fixed client-thread count becomes the bottleneck for larger server configurations. YCSB is evaluated separately at 100, 200, and 400 client threads, corresponding to 100, 200, and 400 concurrent synchronous clients.

Table 3: Role of each load generator.

Tool	Strength	Limitation
<code>redis-benchmark</code>	Broad Redis command coverage	Synthetic fixed-command workloads
<code>memtier</code>	Pipelined mixed workloads with tail latency	Limited Redis data-structure coverage
YCSB	Research-standard application benchmark	Synchronous, non-pipelined Redis access

6.1.1 `redis-benchmark`. The reference C benchmark shipped with Redis [29]. We use it to isolate per-command server-side cost with minimal client-side overhead. This is the only tool in our evaluation that conveniently exercises a wider range of Redis data structures and command types across Redis-compatible systems [28], including SET, GET, HGETALL, ZRANGEBYSCORE, and BITCOUNT. This makes it useful for studying whether THredis accelerates only simple string operations or also generalizes to hash, sorted-set, and bitmap commands.

However, `redis-benchmark` is still a synthetic microbenchmark. Even though it supports pipelining, it does not represent full application traffic: each run focuses on a fixed command pattern rather than a mixed workload with realistic read/write ratios and tail-latency behavior. We therefore use it primarily for command-level analysis rather than as the sole measure of application performance.

For SET and GET, we sweep value sizes {256, 1024} bytes. For HGETALL, we use 50-field hashes. For ZRANGEBYSCORE, we use 100-element sorted sets. For BITCOUNT, we use 1 MiB strings. Pipeline depth P sweeps {1, 16, 32} across all commands. Each configuration uses $-c$ 200 concurrent connections, with the number of client threads matched to the server thread count.

6.1.2 `memtier_benchmark`. We use `memtier_benchmark` to evaluate mixed read/write workloads with pipelined clients. Although it is less commonly used than YCSB in database research papers, it is better suited to the main mechanism evaluated in this work: improving server-side execution under pipelined Redis traffic. Unlike YCSB, `memtier` can issue pipelined request streams, and unlike `redis-benchmark`, it directly supports mixed read/write ratios and reports tail-latency percentiles such as p50, p99, and p99.9.

This makes `memtier` the closest benchmark in our evaluation to a production cache workload, where many clients issue repeated reads and writes against a hot key-value service; prior large-scale key-value workload analysis shows that read/write mix and

object size strongly affect system behavior [2]. Its limitation is that it focuses mainly on string-style key-value operations and does not exercise the broader Redis data structure set covered by `redis-benchmark`. We therefore use `memtier` for production-style pipelined workload behavior, but rely on `redis-benchmark` for command and data-structure breadth.

We drive 200 client connections across N client threads, where N is matched to the server thread count being evaluated. We report two read/write ratios: 1 SET per 10 GETs ($-ratio=1:10$), representing a read-heavy cache workload, and balanced 1:1, representing update-heavy cache or session-store behavior. For both ratios, we sweep value sizes {32, 256, 1024} bytes and pipeline depths {1, 16, 32}.

6.1.3 YCSB. We use the Yahoo! Cloud Serving Benchmark with the `redis` binding as the research-standard application-layer benchmark [8]. YCSB is important because it is widely used in database evaluation and exercises a higher-level client stack, including the JVM driver, garbage collection, connection management, and application-level request generation.

At the same time, YCSB is insufficient for evaluating THredis by itself. The Redis binding follows a synchronous request-response model: each client thread issues a request and waits for the reply before sending the next operation. As a result, it does not generate the pipelined command streams that THredis is designed to accelerate. YCSB therefore evaluates whether THredis remains competitive under conventional application-layer access, but it does not expose the full benefit of execution-level pipelining.

We use the standard workloads:

- **Workload A** (50/50 read-update): exercises mixed read/write behavior.
- **Workload B** (95/5 read-heavy): represents read-dominated cache access.
- **Workload C** (100% read): measures pure lookup throughput.
- **Workload E** (range SCAN with concurrent INSERT): stresses range access and structural mutation.

For each server, YCSB is evaluated at 100, 200, and 400 client threads. Since the Redis YCSB binding uses synchronous request-response, these thread counts also correspond to 100, 200, and 400 concurrent clients. We run each workload on a 2 M-record dataset with Zipfian key distribution for Workloads A–C and uniform distribution for Workload E.

6.1.4 Workload Sweep. The full sweep dimensions are summarized in Table 4. We do not collapse these dimensions into a single best-case number because the same server can be I/O-bound in one cell and execution-bound in another. Reporting the full sweep exposes where each design gains or loses performance.

Table 4: Sweep dimensions per load generator. “Configs” counts the number of server/thread configurations evaluated per workload cell.

Tool	Workload	Pipe.	Scale	Cfg.
redis-bench.	SET/GET	1,16,32	256, 1024 B	18
redis-bench.	HGETALL	1,16,32	50 fields	18
redis-bench.	ZRANGEBYSCORE	1,16,32	100 elems.	18
redis-bench.	BITCOUNT	1,16,32	1 MiB	18
memtier	1:10, 1:1	1,16,32	32, 256, 1024 B	18
YCSB	A/B/C/E	—	100–400 clients	4 sys- tems

The 18 server/thread configurations include the five THredis (10, worker) splits (2i+2w, 2i+4w, 4i+2w, 4i+4w, 6i+4w) at total thread counts 4, 6, 6, 8, and 10, together with the stable Redis, DragonflyDB, and KeyDB sweeps at thread counts {4, 6, 8, 10} each (plus single-threaded Redis for memtier).

6.1.5 Correctness Testing. Performance improvements are only meaningful if the server preserves Redis-compatible semantics [29]. This is especially important for THredis because execution-level pipelining allows commands from the same client connection to be in flight on different worker threads simultaneously. A high-throughput server that returns stale values, incorrect values, nil, or replies in the wrong order is not a valid Redis-compatible implementation.

We therefore evaluate correctness using a dedicated pipelined correctness harness. The harness creates multiple logical clients across POSIX threads; each client opens its own Redis connection, issues deterministic pipelined command sequences, drains replies in order, and checks the exact Redis reply type and value. To avoid false failures from intentional cross-client races, each client uses a private key namespace of the form <pid>:c<CID>:<category>:<K>. Therefore, any mismatch indicates an implementation error in command execution, reply ordering, or per-client pipeline semantics rather than a race-dependent artifact of the test.

The harness covers the same command classes used in our benchmarks, including strings, counters, bitmaps, hashes, lists, sets, sorted sets, and miscellaneous commands. Example checks include SET/GET, INCR/GET, SETBIT/BITCOUNT, HSET/HGET, RPUSH/LINDEX, SADD/SISMEMBER, and ZADD/ZSCORE. The expected reply is computed from the issued command order, and the checker verifies status replies, integer replies, bulk strings, and nil values.

We include two ordering-specific tests for THredis’s execution model. First, the pipeline reply-ordering test targets out-of-order reply flushes. A client issues a single pipeline batch containing commands that may be dispatched to different workers and complete at different times. The test passes only if the client receives the replies in the exact order in which the commands were issued, regardless of their internal completion order. This checks that THredis’s reply path behaves like in-order retirement rather than flushing worker completions as soon as they finish.

Second, the same-key commit-order test targets program-order visibility. For each key, the client issues a sequence of dependent operations, such as multiple overwrites followed by a GET, in one pipeline batch. The younger read must observe the effects of the older writes, and the final GET must return the value from the last write in client program order. This checks that out-of-order

execution does not allow a later same-key operation to bypass an earlier same-key update.

We also ran the correctness harness while the server was simultaneously saturated by independent redis-benchmark workloads. This tests whether the same guarantees hold under queue pressure, worker contention, and reply-flush pressure, rather than only in an isolated correctness run. All server configurations reported in the results section passed both the standalone correctness harness and the correctness-under-load run with zero failed checks.

The default correctness run uses 100 logical clients, 100 keys per client, and pipeline depth 16, producing approximately 5.4 million checked replies. The harness is fail-fast and uses no wall-clock sleeps or timing-based assertions; a passing run means every checked reply matched the Redis-visible result implied by the command sequence.

7 Evaluation

We evaluate THredis against Redis, KeyDB, and DragonflyDB across pipelined and synchronous workloads; these baselines represent Redis’s canonical single-execution-thread design, KeyDB’s multi-threaded Redis fork, and DragonflyDB’s shared-nothing/proactor architecture [10, 11, 27, 29, 32]. The evaluation answers five questions: (1) whether execution-level pipelining improves throughput, (2) how the benefit changes with pipeline depth, payload size, and thread count, (3) whether the design helps beyond GET and SET, (4) what THredis costs when clients do not pipeline requests, and (5) where each server spends CPU time. The workloads, sweep dimensions, client-thread policy, and correctness suite are defined in Section 6; this section focuses on results and interpretation.

7.1 Setup

All experiments run on a single AMD Ryzen 7 7700X system (8 physical cores, 16 SMT threads, 4.5 GHz base) with 64 GB DDR5-5200 memory, running Linux 6.17. We compare against stable Redis 8.6.2 – the upstream version from which THredis is forked – KeyDB 6.3.4, and DragonflyDB 1.38. All three baselines are evaluated with their default configurations except where noted (see the DragonflyDB encoding sanity-check below). Under the matched client-thread policy defined in Section 6, 6- and 8-thread server configurations use SMT contexts and 10-thread server configurations oversubscribe the 16 hardware threads. We therefore interpret these points as end-to-end saturation measurements, not isolated server-only scaling. We additionally verified DragonflyDB measurements with its default storage encoding disabled and observed no significant change in throughput; all results are reported with default configurations for each server.

7.2 Headline pipelined throughput

THredis delivers substantial throughput gains over stable Redis on pipelined workloads while preserving competitive tail latency, with the advantage growing as the machine is more heavily loaded. We establish those gains here at the canonical operating point—pipeline depth 16, 32-byte values, balanced and read-heavy mixes—and study sensitivity to pipeline depth, payload size, and command type in subsequent subsections.

Using the `memtier_benchmark` setup from Section 6, the headline workload uses 32-byte values and pipeline depth 16. We report the 1:1 and 1:10 read/write mixes separately because they stress THredis and the baselines in different ways.

7.2.1 Balanced headline workload (1:1). Figure 7a reports the main balanced result. At eight server threads, THredis achieves **4.99 M ops/sec** versus **3.11 M** for stable Redis, a 1.60× advantage, while keeping p99 latency essentially unchanged relative to Redis (**1.29 ms vs 1.33 ms**). This point is already a full-machine saturation point: the eight server threads are paired with eight client threads, filling the 16 SMT contexts on the 8-core machine. Extending to ten server threads creates 20 runnable client+server threads, so the benchmark includes physical-core contention; under this pressure the throughput gap widens to **1.98× (5.73 M vs 2.89 M)**, but p99 latency rises to **3.04 ms**. We therefore treat the 10-thread point as a throughput-oriented oversubscription result, not a lower-latency operating point.

The widening reflects two facts visible in the same figure. First, Redis’s throughput *decreases* from 3.48 M (4 threads) to 3.11 M (8 threads) to 2.89 M (10 threads): adding helper threads beyond four produces negative returns because Redis’s single execution thread is already saturated. The profile data in §7.9 confirms this. Second, THredis’s hash-partitioned workers continue to add useful parallelism: 4.99 M → 5.73 M from 8 to 10 threads. The latency curve shows the cost of pushing further: THredis’s p99 is best at the 8-thread full-machine point and increases once the run is oversubscribed. Thus, the headline result is not simply “more threads are always better.” The clean operating point is 8 server threads; the 10-thread result shows that THredis can still extract more throughput under extra contention, but with a tail-latency cost.

7.2.2 Read-heavy headline workload (1:10). The right panel of Figure 7 reports the same sweep at 1:10. The curves have similar shapes, but the relative gaps change because read-heavy traffic makes Redis’s single execution path much cheaper. At eight threads, THredis reaches **5.12 M ops/sec** versus Redis’s **4.06 M**, with p99 latency of **1.23 ms** versus Redis’s **1.00 ms**.

Read-heavy traffic helps Redis the most. Going from 1:1 to 1:10 at 8 threads, stable Redis gains **31%** throughput (3.11 M → 4.06 M); the multi-threaded designs gain only 2–4%. Redis benefits because GET is much cheaper than SET: it avoids allocation, AOF append-buffer writes, and replication-stream pushes. The other servers already spread request processing across cores, so switching to mostly reads gives them less extra headroom.

The competitive gap narrows on read-heavy traffic. At 8 threads on the 1:10 mix, the cross-system picture is:

Server	1:1 vs Redis	1:10 vs Redis
THredis	1.60×	1.26×
DragonflyDB	1.34×	1.05×
KeyDB	0.78×	0.63×

THredis’s advantage shrinks from 60% to 26%; DragonflyDB falls to essentially ties with Redis (1.05×); only KeyDB does not improve its position. The implication is that multi-threading in a Redis-compatible server pays off most clearly on *conflict-heavy* workloads (mixed read/write) and least on read-heavy traffic where the serial

path is already efficient. THredis preserves its lead at 1:10 because its parallelism advantage comes from execution-level pipelining (which is present at any read/write ratio), not from spreading write-side overhead across threads.

The 4-thread cell is THredis’s least favorable configuration. On the 1:10 mix at 4 threads, stable Redis (4.56 M) edges out THredis (3.42 M). Each worker has only ~2 cores of execution capacity behind it, and the dispatch hop is a larger fraction of total work. From 6 threads onward THredis takes the lead and holds it; at 10 threads, the gap re-opens to **1.63× (6.14 M vs 3.76 M)** as Redis again loses ground under SMT and scheduling pressure. As in the 1:1 case, this 10-thread gain comes with worse tail latency: THredis’s p99 rises to **3.12 ms**, while Redis remains near **0.99 ms**.

7.3 Server-thread scaling efficiency

We next normalize the headline throughput by server thread count. Under the client-pressure rule defined in Section 6, this shows how much end-to-end throughput each added server thread buys.

7.3.1 The scaling curves (1:1). Figure 8a reports throughput divided by server thread count as the server configuration grows from 4 to 10 threads on the 1:1 mix. Because the matched-client policy is already defined in Section 6, the curves show end-to-end scaling under matched client pressure. Each curve’s shape matches the bottlenecks identified in §7.9.

- **Stable Redis** collapses from 870 K ops/sec per server thread at 4 threads to 289 K at 10 threads – a 67% efficiency loss. The single execution thread saturates and additional helper threads contribute negative returns, as confirmed by the asymmetric main-thread vs IO-thread load reported in §7.9.
- **KeyDB** barely scales: 529 K → 291 K per server thread (45% loss). Adding server threads adds `fastlock_lock` contention, partially canceling the parallelism benefit.
- **DragonflyDB** degrades gracefully: 751 K → 395 K per server thread (47% loss). Per-shard locality remains good, but the per-completion kernel cost grows as a fraction of total work as server threads are added.
- **THredis** shows the gentlest decline when properly tuned: 819 K → 573 K per server thread, a 30% efficiency loss. Cache-coherence cost grows with parallelism, but the parallel work itself remains productive (validated by the hot-path analysis of §7.9).

7.3.2 Configuration sensitivity at 6 server threads. The dashed purple detour at 6 server threads in Figure 8a shows the worse THredis split *for this workload* (2i+4w) at 472 K ops/sec per server thread – *below* DragonflyDB’s 597 K and only marginally above Redis’s 572 K. The well-tuned 4i+2w configuration at the same total budget delivers 708 K ops/sec per server thread, restoring the lead shown by the solid line. As §7.6 demonstrates, “well-tuned” depends on the workload: 4i+2w is correct for short commands like the GET/SET mix in this figure, while 2i+4w would be correct for compute-heavy commands like HGETALL or ZRANGEBYSCORE. The throughput sensitivity is real either way; only at 4, 8, and 10 server threads is the (io,worker) split fixed by our chosen configurations.

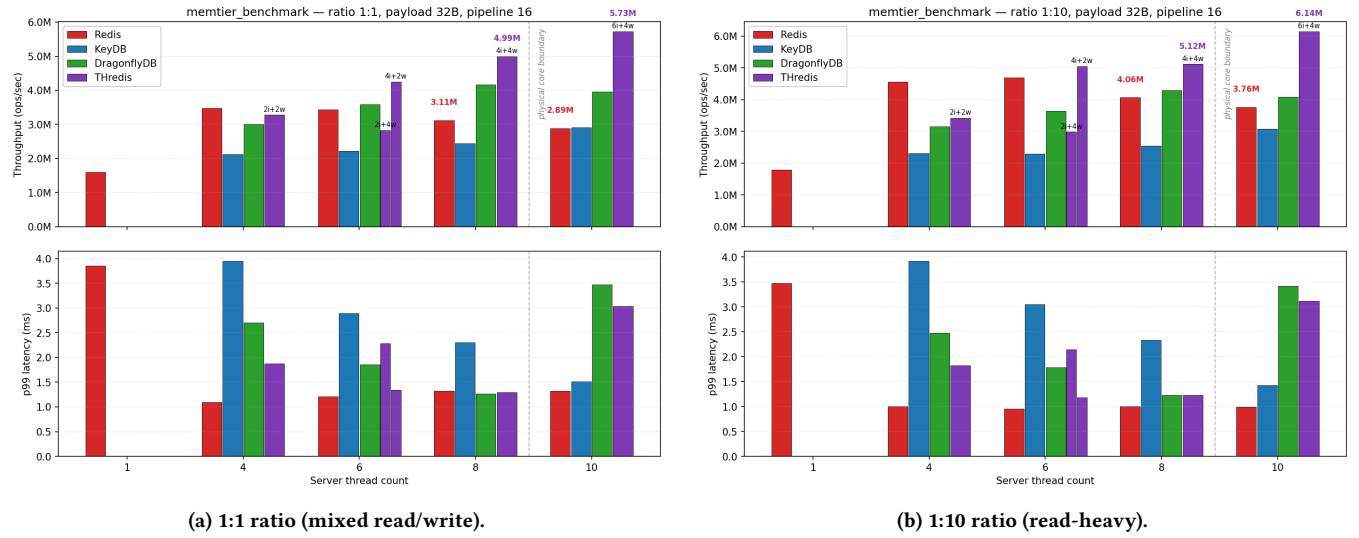


Figure 7: Throughput (top) and p99 latency (bottom) versus server thread count, memtier_benchmark, 32 B values, pipeline 16. The matched client-thread policy is defined in Section 6. THredis’s two (io_threads, worker_threads) splits at 6 server threads are shown side-by-side within each panel.

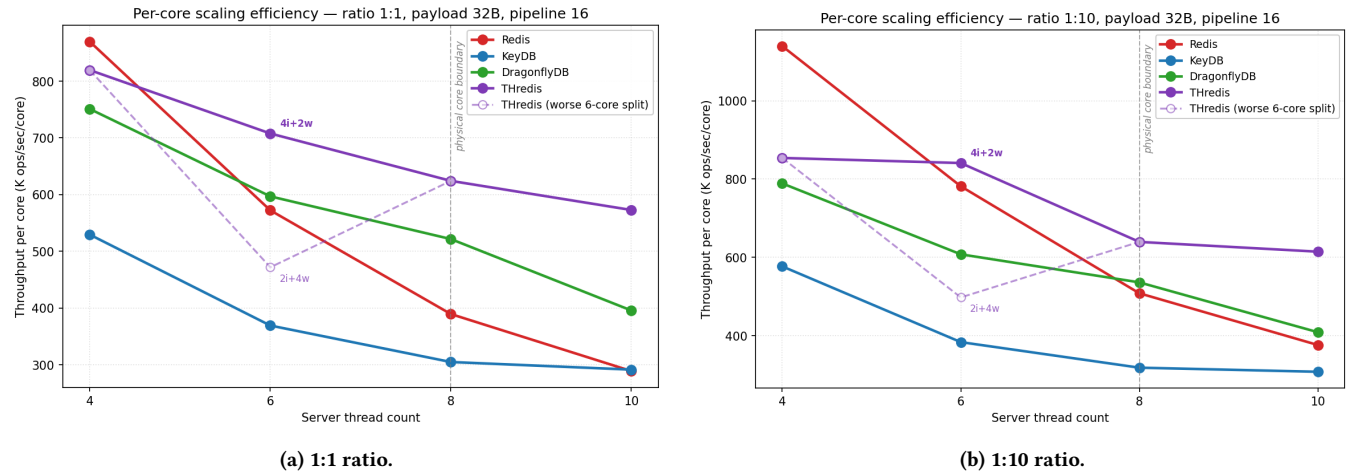


Figure 8: Throughput normalized by server thread count, 32 B values, pipeline 16. The matched client-thread policy is defined in Section 6. In the 1:1 panel, the dashed purple detour at 6 server threads shows THredis’s worse (io, worker) split (2i+4w); the solid line uses the better 4i+2w configuration.

7.3.3 The same shape on read-heavy traffic. The 1:10 mix tells the same story with absolute numbers shifted upward (Figure 8b, the right panel). THredis maintains 614 K ops/sec per server thread at 10 threads while Redis drops to 376 K, a $1.63\times$ normalized-throughput advantage. KeyDB and DragonflyDB sit between, both with similarly steep declines from their 4-thread baselines.

7.3.4 The main claim. The ranking at 10 threads – THredis > DragonflyDB > Redis $\approx$ KeyDB on throughput normalized by server thread count – summarizes the main claim of this paper: under

pipelined cache workloads at scale, single-writer partitioning produces a more favorable scaling curve than single-threaded serialization, per-key locking, or shared-nothing multiplexing.

7.4 Pipeline-depth sensitivity and synchronous-load cost

The headline result assumes pipelined clients. We now vary pipeline depth to show where that benefit appears, and then test the opposite case: synchronous clients that issue one request at a time.

7.4.1 Pipeline-depth crossover. Figure 9 sweeps pipeline depth at eight server threads and 32 B values. At pipeline depth 1, THredis is slower than stable Redis on both mixes: 0.62 M vs 0.85 M on 1:1, and 0.64 M vs 0.85 M on 1:10. This is the expected cost of the design. Each request pays the dispatch hop and fake-client migration cost, but there is no batch of later commands to spread that cost across.

Once clients pipeline enough commands, the trade-off reverses. At pipeline 16, THredis is 1.60× ahead of Redis on 1:1 (4.99 M vs 3.11 M) and 1.26× ahead on 1:10 (5.12 M vs 4.06 M). At pipeline 32, the lead grows to 6.59 M vs 2.71 M on 1:1 and **7.18 M vs 3.65 M** on 1:10. The mechanism is the same in both cases: a deeper pipeline gives workers enough independent commands to execute in parallel, while the IO thread can drain multiple completed replies together.

KeyDB shows the opposite behavior: it underperforms stable Redis at every pipeline depth on both mixes because its lock-acquisition cost is paid on every command and batching does not remove it. DragonflyDB tracks more closely with THredis from pipeline 16 onward, and at pipeline 32 both substantially exceed stable Redis.

7.4.2 Synchronous clients remove THredis’s main advantage. As defined in Section 6, YCSB uses synchronous Redis access; YCSB remains the standard application-layer benchmark for cloud-serving systems [8]. It therefore behaves like the pipeline-depth-1 case at a larger client count and directly tests the cost of THredis when execution-level pipelining is absent. Figure 10 reports YCSB throughput at 200 client threads.

Workload	Redis	KeyDB	DragonflyDB	THredis
A (50/50)	517.6 K	352.5 K	487.6 K	358.4 K
B (95/5)	467.9 K	329.3 K	478.9 K	394.6 K
C (100%R)	468.4 K	340.0 K	476.6 K	403.2 K
E (scan)	9.1 K	6.2 K	10.6 K	8.8 K

THredis loses to stable Redis by **31%** on workload A, **16%** on B, **14%** on C, and nearly ties on E (4% lower). This is the design’s main cost. Without a pipelined batch, the dispatch hop is pure overhead rather than the price paid for parallel in-flight execution. The implication is clear: THredis targets pipelined cache workloads, not synchronous one-request-at-a-time clients.

DragonflyDB is strongest under this synchronous model. It beats stable Redis on workloads B, C, and E, and is within 6% on A, because per-shard ownership lets synchronous requests execute without THredis’s cross-thread dispatch path. KeyDB is uniformly worst, 27–32% behind Redis on A–C and 32% behind on E, consistent with the lock-cost results in §7.9.

7.5 Workload generality: payload size and data structures

We next vary the workload beyond GET/SET ratio and pipeline depth. The same section covers value size, bandwidth limits, and commands on Redis data structures beyond simple strings.

7.5.1 Payload size. Once clients pipeline, THredis’s advantage holds across the payload sizes covered by typical cache workloads. On the redis-benchmark sweep at eight server threads and pipeline 16, THredis leads stable Redis on every measured GET and SET cell: at 256 B by 1.44× on GET (5.00 M vs 3.47 M) and 1.33×

on SET (3.81 M vs 2.86 M); at 1024 B by 1.29× on both GET (3.81 M vs 2.96 M) and SET (3.33 M vs 2.58 M). At pipeline 32 the 1024 B advantage widens to 1.50× on GET (4.44 M vs 2.96 M) and 1.40× on SET (3.20 M vs 2.28 M). At ten server threads (6i+4w), 1024 B redis-benchmark reaches 4.20 M GET ops/sec versus Redis’s 2.96 M (1.42×). THredis also stays ahead of KeyDB and DragonflyDB on every 1024 B pipelined cell measured. The memtier mixed-ratio sweep at 32 B and 256 B agrees with this picture: THredis is 1.60× ahead at 32 B (4.99 M vs 3.11 M) and 1.15× ahead at 256 B (4.30 M vs 3.74 M) on the 1:1 mix, with similar ratios on 1:10. The result is a consistent pipelined-throughput lead over stable Redis from 32 B through 1 KiB payloads, and a consistent lead over KeyDB and DragonflyDB across the same range.

This payload sensitivity to pipelining is consistent with prior large-scale key-value workload analysis, where object size and request mix interact with the dominant bottleneck [2]—and where pipelining, when present, is what lets the server amortize per-request overhead across the batch.

Command	Redis	KeyDB	DragonflyDB	THredis
SET (256 B)	2.86 M	1.05 M	3.63 M	3.81 M
GET (256 B)	3.47 M	1.51 M	1.18 M	5.00 M
SET (1024 B)	2.58 M	0.84 M	2.76 M	3.33 M
GET (1024 B)	2.96 M	1.08 M	0.99 M	3.81 M
HGETALL (50 fields)	0.39 M	0.25 M	0.71 M	1.01 M
ZRANGEBYSCORE (100)	0.28 M	0.32 M	0.75 M	0.69 M
BITCOUNT (1 MiB)	3.99 M	3.99 M	4.99 M	5.00 M

THredis gains most when each command has real CPU work to parallelize *and* the request stream is pipelined. HGETALL is the clearest case: 1.01 M for THredis versus 0.39 M for Redis, a 2.59× improvement. SET and GET on 256 B values show 1.33–1.44× gains; the same commands at 1024 B show 1.29× gains, confirming that the regime extends through 1 KiB payloads rather than dropping off as bytes-per-command grows. The result supports the main mechanism: THredis helps when the server is CPU-bound and the pipelined batch contains enough independent work, and the regime covers the payload sizes that dominate typical cache workloads.

The exceptions are informative. ZRANGEBYSCORE is the only command where DragonflyDB clearly beats THredis (0.75 M vs 0.69 M), likely because DragonflyDB’s sorted-set implementation is more cache-efficient than the inherited Redis codepath that THredis uses unmodified. BITCOUNT on 1 MiB values is the bandwidth-bound limit case: all four servers cluster between 4.0 M and 5.0 M ops/sec because the 10 GbE link dominates. THredis is not meaningfully faster here, but it is also not worse than the multi-threaded alternatives.

The comparison also exposes architecture-specific weaknesses. DragonflyDB is competitive on SET, ZRANGEBYSCORE, and BITCOUNT, but its GET throughput is only 1.18 M, below stable Redis and far below THredis; the profile in §7.9 shows that short reads expose a large kernel-side cost in DragonflyDB’s per-Proactor io_uring path [10]. KeyDB is worst on every command except ZRANGEBYSCORE, and on HGETALL it falls below single-threaded Redis because fastlock_lock dominates CPU time.

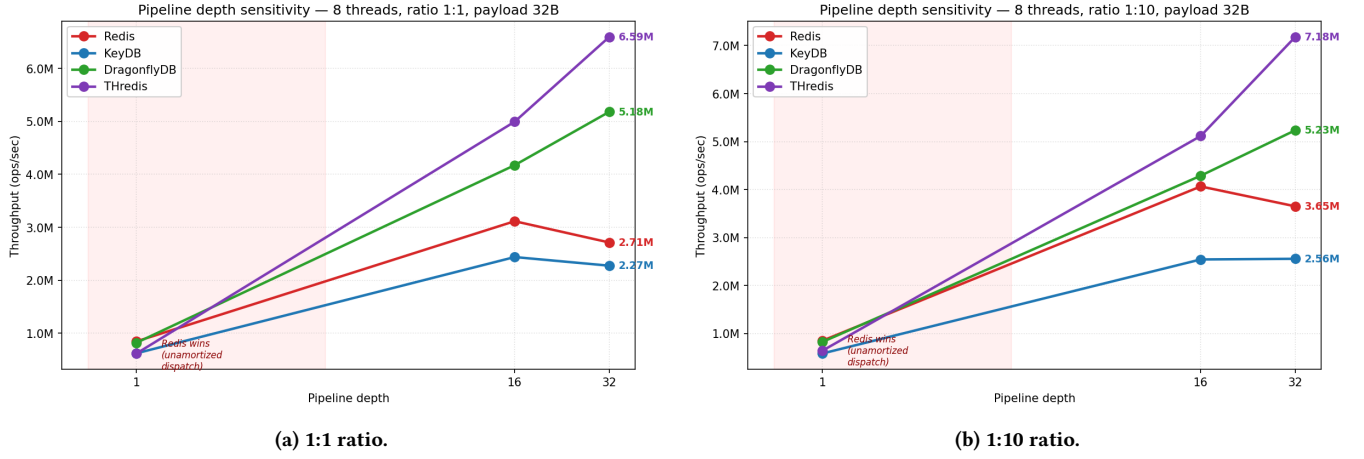


Figure 9: Throughput versus pipeline depth at eight server threads, 32 B values. Stable Redis wins at $P = 1$; THredis wins from $P = 16$ onward on both mixes.

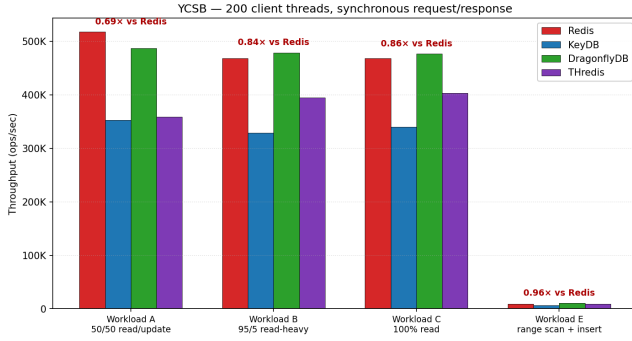


Figure 10: YCSB throughput at 200 client threads. The synchronous client model defined in Section 6 removes THredis's pipelining advantage.

7.6 IO/worker thread budget

An operator deploying THredis must choose how to divide a thread budget between IO threads and worker threads. We characterize the best split.

7.6.1 Split sensitivity at low thread budgets. The 6-thread cell in Figures 7a and 7b shows both (io_threads, worker_threads) splits side by side. At 1:1, 32 B, pipeline 16, the 4i+2w split delivers 4.25 M ops/sec while the 2i+4w split delivers only 2.83 M, a **50% throughput difference** between two configurations using the same total thread budget. The split sensitivity is even more pronounced on the 1:10 mix: 4i+2w hits **5.05 M** versus 2i+4w's 2.98 M, a 69% gap. The 4i+2w configuration at 6 server threads nearly catches 4i+4w at 8 server threads (4.25 M vs 4.99 M on 1:1).

7.6.2 Interpretation. The split sensitivity on the short-payload memtier workload indicates that the IO path is the bottleneck at low total budgets: request parsing, reply emission, and the IO event loop consume a disproportionate share of work, and adding a second IO thread is more valuable than adding a second worker. At 8+ threads

the split sensitivity vanishes and IO/worker contributions roughly balance.

This explanation is correct for *short commands like GET and SET* where per-request execution is cheap. The story changes substantially for commands with heavier per-request work, as the next subsection shows.

7.6.3 Compute-heavy commands prefer more workers. redis-benchmark at 6 server threads gives us the same dual-split data across a wider command set, and the pattern is markedly different from memtier's GET/SET-only mix:

Command	4i+2w	2i+4w	Winner	Gap
GET (256 B)	3.64 M	2.76 M	4i+2w	1.32×
SET (256 B)	2.76 M	2.16 M	4i+2w	1.28×
HGETALL	0.60 M	0.93 M	2i+4w	1.56×
ZRANGEBYSCORE	0.43 M	0.69 M	2i+4w	1.61×
BITCOUNT (1 MiB)	5.00 M	2.85 M	4i+2w	1.75×

The best split depends on the command type. Short string commands (GET, SET) prefer more IO threads – 4i+2w beats 2i+4w by 28–32%. The IO path is the bottleneck because per-request execution is cheap and the worker finishes its share of the batch before the IO thread has parsed the next batch.

Compute-heavy commands (HGETALL, ZRANGEBYSCORE) prefer more workers – 2i+4w beats 4i+2w by 56–62%. Per-request execution is now expensive enough that the worker side becomes the bottleneck; two workers cannot keep up with four IO threads' incoming command rate, and adding worker capacity is what unblocks throughput.

Bandwidth-bound workloads (BITCOUNT on 1 MiB values) revert to favoring IO threads (4i+2w 1.75× better) because the network egress path – assembling and emitting large reply buffers – is mostly an IO-side cost.

Deployment rule. The single-knob heuristic of §7.6.2 ("favor IO threads at low budgets") is correct for the throughput-main case

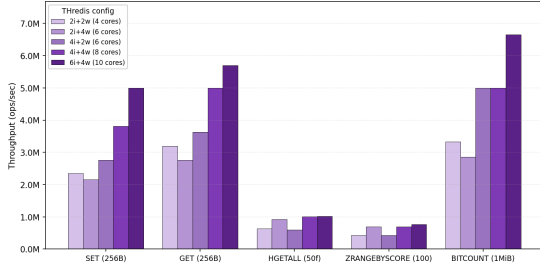


Figure 11: THredis throughput across all measured (io_threads, worker_threads) configurations for five command types at pipeline 16. Bars within each command cluster, left to right, show: 2i+2w (4 server threads), 2i+4w (6 server threads), 4i+2w (6 server threads), 4i+4w (8 server threads), and 6i+4w (10 server threads). Annotations above each cluster show the percentage gain from 4i+4w to 6i+4w – a transition that adds two IO threads with no new workers. SET, GET, and BITCOUNT scale by 14–33%; HGETALL barely moves (+1%); ZRANGEBYSCORE adds 10%.

of short key-value workloads but wrong for compute-heavy commands. A better rule for low-budget configurations: *tune the split toward whichever side has higher per-request cost*. If the deployment is dominated by GET, SET, or other simple commands, IO threads should slightly outnumber workers. If the deployment is dominated by HGETALL, ZRANGEBYSCORE, or other commands with substantial per-request CPU work, workers should slightly outnumber IO threads. At 8+ threads the split sensitivity vanishes and balanced splits perform well across the command spectrum.

Possible automatic tuning. An automatic configuration mechanism could observe the workload’s command mix and re-tune the split dynamically; THredis does not currently implement this, but the data suggests it would deliver 30–60% throughput improvements on the workloads where the static configuration is mismatched. We leave this as future work.

7.6.4 Adding only IO threads does not help worker-bound workloads. The 8-thread (4i+4w) to 10-thread (6i+4w) transition is a particularly clean experiment because it adds two IO threads without changing the worker count. Figure 11 plots THredis throughput at every measured (io, worker) configuration across all five command types at pipeline 16, and the 4i+4w → 6i+4w delta is annotated above each command cluster.

The split sensitivity confirms the previous explanation. On SET, the IO-only addition delivers a 31% throughput gain (3.81 M → 4.99 M); on GET, 14%; on BITCOUNT, 33%. These are commands where the IO path is at or near the bottleneck, so adding IO capacity relieves the bottleneck. On HGETALL, the same IO-only addition delivers **only 1%** (1.01 M → 1.02 M) – effectively no scaling at all. Four worker threads are already saturated on this workload, and adding more IO capacity simply piles up commands waiting on workers. ZRANGEBYSCORE shows an intermediate 10% gain because its per-request cost is heavy enough to bottleneck workers but not enough to fully saturate them.

Table 5: Latency percentiles on mentier_benchmark, 1:1 ratio, 32 B values, pipeline 16. p50/p99/p999 are in milliseconds; rps is in millions.

Server	rps	p50	p99	p999	p99/p50
4 server threads					
Redis	3.48	0.92	1.09	1.62	1.2×
KeyDB	2.12	1.36	3.95	4.64	2.9×
DragonflyDB	3.00	0.96	2.70	3.68	2.8×
THredis	3.28	0.92	1.88	2.00	2.0×
8 server threads					
Redis	3.11	1.01	1.33	2.05	1.3×
KeyDB	2.44	1.18	2.30	2.69	1.9×
DragonflyDB	4.17	0.76	1.27	1.70	1.7×
THredis	4.99	0.58	1.29	2.06	2.2×
10 server threads					
Redis	2.89	1.12	1.33	2.38	1.2×
KeyDB	2.91	1.14	1.51	1.81	1.3×
DragonflyDB	3.95	0.67	3.47	4.29	5.2×
THredis	5.73	0.38	3.04	4.42	8.1×

This is a useful signal for capacity planning. If an operator measures their workload at the main 4i+4w and observes that adding IO threads does not increase throughput, the result shows that workers are the bottleneck. The fix is either adding worker threads or shifting workload toward less compute-heavy commands; adding IO capacity in this case is wasted hardware. The opposite holds when adding IO threads does scale: the workers are not the bottleneck and IO capacity is the right place to invest.

The 6-server-thread data also reflects this. Within the 6-server-thread budget, 2i+4w (6 server threads total) reaches 0.93 M on HGETALL – higher than 4i+4w’s 1.01 M at 8 server threads despite using fewer total server threads, and 92% of 4i+4w’s absolute throughput at 75% of the thread budget. For HGETALL workloads, 2i+4w at 6 server threads is the most efficient configuration tested, beating both higher-budget 4i+4w and the IO-heavy 4i+2w at the same 6-server-thread total.

7.7 Latency under load

We measure how queueing and in-order retirement affect median and tail latency.

Table 5 reports p50, p99, and p999 latencies at three operating points. The data complicates the simple narrative that THredis trades latency for throughput: median latency favors THredis under load, while tail amplification rises once client and server threads share SMT contexts and, at 10 server threads, oversubscribe the machine.

7.7.1 Median latency favors THredis. At 8 threads, THredis’s p50 is **0.58 ms**, the lowest of any server tested and 42% below stable Redis’s 1.01 ms. At 10 threads the gap widens to 0.38 ms vs 1.12 ms. The reason is queueing discipline: at saturation, stable Redis serializes all in-flight commands behind its single execution thread, forcing even simple GETs to wait behind earlier work; THredis’s hash-partitioned workers reduce the effective queue depth seen by any individual command to a per-shard quantity. This produces a

counterintuitive result – a multi-threaded design with a measurable per-request dispatch overhead nonetheless delivers lower median latency than a single-threaded design under load – because the dispatch overhead is dominated by the queuing relief that parallel execution provides.

7.7.2 p99 is competitive at eight server threads. At 8 threads, THredis’s p99 of 1.29 ms is statistically equivalent to stable Redis’s 1.33 ms while delivering 1.60× throughput. DragonflyDB’s p99 of 1.27 ms is comparably tight. KeyDB’s tail is consistently worst across all configurations: 2.30 ms at 8 threads, 3.95 ms at 4 threads. The lock-induced serialization at the heart of KeyDB’s architecture causes worse tail behavior than either pure single-threaded serialization or fake-client queuing.

7.7.3 Tail amplification rises under shared-core pressure. At 10 threads, THredis’s p99/p50 ratio jumps to 8.1× (0.38 ms median, 3.04 ms 99th percentile) – the highest of any server. This is the visible cost of in-order retirement: when any concurrent execution in a pipeline batch is slow (due to SMT sharing, scheduler latency, or memory-system interference), all later replies in the batch wait for it before the drain can advance. The design preserves the in-order reply guarantee that Redis clients depend on, so this cost is caused by the design rather than accidental. DragonflyDB exhibits a similar but smaller effect under the same oversubscription (5.2× amplification). At 8 server threads, the amplification ratios are tighter (THredis 2.2×, DragonflyDB 1.7×, Redis 1.3×); the oversubscription at 10 server threads is what stretches the THredis tail.

7.7.4 Latency on synchronous workloads. YCSB latencies tell the same story as YCSB throughput. THredis’s read latency on workload A at 200 client threads is 432 μ s versus Redis’s 373 μ s and DragonflyDB’s 379 μ s, reflecting the dispatch hop’s inherent overhead when there is exactly one in-flight request per client. KeyDB sits at 531 μ s on the same workload. Under synchronous access, THredis’s queueing-relief advantage disappears, and the dispatch hop becomes purely additive. This is the same trade-off visible in §7.4: THredis’s design optimizes for pipelined load and pays a small cost when pipelining is absent.

7.7.5 In-order retirement. The in-order retirement guarantee adds latency cost equal to the slowest concurrent execution in a batch. A reviewer might ask whether out-of-order retirement would help; it would, but the design’s value depends on preserving client-visible reply order, so this is a deliberate choice rather than an oversight.

7.8 Per-request CPU cost

We quantify per-request CPU efficiency in cycles per request, the metric that shows how much CPU work each request costs.

7.8.1 Cycles per request. Figure 12 reports cycles per request across five workloads, computed as the cycle count from a 30-second perf stat window divided by requests served during that window. Stable Redis is the most cycle-efficient on every workload: single-threaded execution avoids the cache-coherence and queue-ordering costs that any multi-threaded design must pay. THredis spends 1.4–1.9× more cycles per request than Redis (7 846 vs 5 469 on GET; 30 117 vs 18 118 on HASH), reflecting the dispatch hop and cross-thread fake-client migration. DragonflyDB’s per-request cost is higher

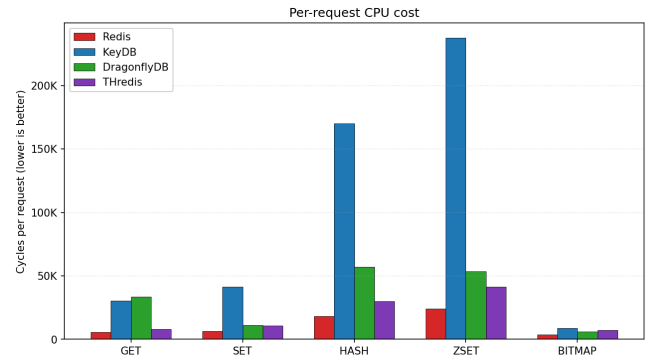


Figure 12: Cycles per request across five workloads at eight server threads, computed from perf stat hardware counters.

than THredis’s on GET specifically (33 676 vs 7 846), an anomaly we attribute in §7.9.

7.8.2 KeyDB cycle cost. KeyDB stands apart: 30 K–237 K cycles per request, 5–10× Redis’s cost. This aligns with KeyDB’s design choice to expose multi-threaded command execution over a shared Redis-compatible dataset [32]. Decomposing this with the symbol-level data of §7.9, almost all of the gap is fastlock_lock; KeyDB’s actual command-execution code is within 30% of Redis’s per-request cost, but the lock acquisition path accounts for most of the work it was meant to serialize.

7.8.3 Throughput math. THredis’s per-request cycle cost is higher than Redis’s, but those cycles execute in parallel across worker threads that Redis’s design cannot usefully use. At ten server threads on the GET workload, THredis converts approximately 45 G thread-cycles per second into useful work (5.73 M rps × 7 846 cycles/req); stable Redis converts approximately 16 G thread-cycles per second because its main thread saturates at four IO helpers (§7.2). The 1.98× throughput advantage comes from this trade-off: THredis spends about 1.4× more cycles per request, but it devotes about 3× more cycles to useful parallel work.

7.8.4 Instruction count. Instructions per request separate extra executed code from memory and coordination stalls. By this measure, THredis is within 5% of Redis’s instruction count on every workload (e.g., 6 404 vs 6 688 on GET; 54 775 vs 55 266 on HASH). The dispatch hop adds little instruction-level work; the cycle gap is dominated by memory-subsystem and coordination cost, not extra computation.

7.8.5 IPC as signal. Instructions-per-cycle reveal where each design’s cycles go. Redis’s IPC of 1.22–3.40 reflects the single-threaded execution path with no coordination overhead. KeyDB’s IPC of 0.27–0.64 indicates cores stalling 40–75% of the time, the main pattern of contended spinlocks, and explains the large cycles/req cost. THredis’s IPC of 1.82–2.08 on hash and sorted-set commands matches DragonflyDB’s; on GET and SET, THredis sits below DragonflyDB because the dispatch hop is more visible when each command does little work.

7.9 Cycle breakdown: where does each server spend time?

The library and symbol breakdowns reveal each design’s bottleneck pattern directly. We collected 15-second symbolicated perf record samples on each server at eight threads.

Figure 13a shows the CPU breakdown by library across five workloads, and Figure 13b reports the single hottest function per (server, workload) cell. Each row of the table shows a different design bottleneck.

7.9.1 Stable Redis: kernel-bound on small reads. On the GET workload, the single hottest function is the kernel `_copy_from_iter` at 17.5% of CPU – the per-request data copy through the socket layer, executed once per command because Redis emits replies one at a time. Total kernel time is 55.3% on GET. THredis’s consolidated drain (§4.6) reduces this to a single `sendmsg` per pipeline batch, and `_copy_from_iter` does not appear among THredis’s top-30 hot symbols. The shift in hottest function – from a per-request kernel copy in Redis to a per-batch userspace worker loop in THredis – is the syscall consolidation claim shown in the profile.

We additionally observe that Redis’s IO-thread distribution is asymmetric: at 8 threads on the SET workload, the main `redis-server` thread takes 28.4% of CPU while each of the seven `io_thd_N` helpers takes only ~10%. The main thread carries the entire command-execution path while helpers offload only IO; this empirically explains why Redis’s throughput per server thread collapses beyond four threads (§7.3).

7.9.2 KeyDB: dominated by lock acquisition. A single function, `fastlock_lock`, accounts for 60–85% of CPU across every workload (GET 77.2%, SET 76.0%, HASH 84.6%, ZSET 77.1%, BITMAP 59.4%). KeyDB’s actual command-execution code accounts for less than 10% of cycles. The lock-acquisition path is 3–7× larger than the command work it protects. KeyDB’s design tradeoff – multi-threaded execution protected by per-key spinlocks – does not pay off here because the lock cost is larger than the command cost. KeyDB’s high userspace fraction in Figure 13a (89–95% across most workloads) is not a sign of an efficient design; it indicates cores spinning on locks in userspace rather than doing useful work. This is corroborated by the IPC collapse to 0.27–0.64 reported in §7.8.

7.9.3 DragonflyDB: kernel-bound on small reads. DragonflyDB’s GET cost is dominated by per-request kernel overhead. On GET, DragonflyDB spends only 29.4% of CPU in its own userspace and **68.5% in the kernel**. Within the kernel, the costs distribute across the TCP/IP stack (~13% combined across `tcp_sendmsg_locked`, `_copy_from_iter`, `__tcp_transmit_skb`, `tcp_v4_rcv`, and related functions), netfilter connection tracking (~3%), AMD SRSO microcode mitigations (~3%, present in all servers but exposed by DragonflyDB’s high syscall rate per useful work), and per-syscall housekeeping (~4%). The largest userspace function is `ascii_unpack` (value decoding) at 3.28%; we verified that disabling DragonflyDB’s storage encoding produces no measurable change in GET throughput, ruling out decode cost as a primary contributor. The per-Proactor distribution is uniformly $12.5\% \times 8$, ruling out load imbalance.

The pattern is consistent with DragonflyDB’s per-Proactor `io_uring` path paying a fixed kernel-side cost per completion that

does not get hidden when per-request userspace work is small. SET avoids this case: userspace work expands to 74.8% of CPU (DashTable insertion, AutoUpdater bookkeeping, allocation tracking), and the same kernel cost becomes a smaller share of total work.

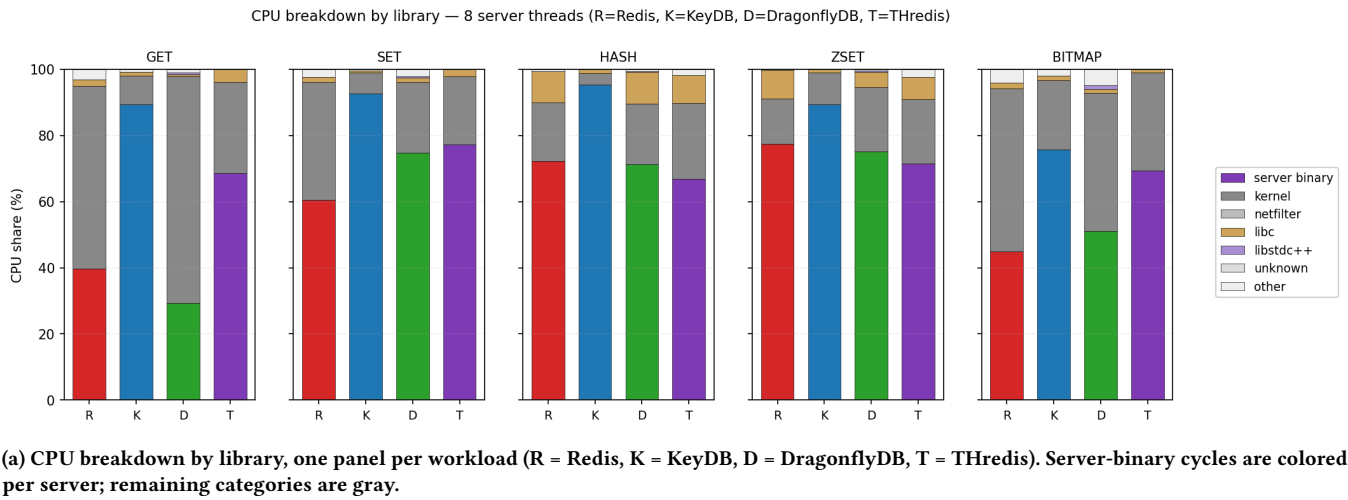
7.9.4 THredis: hot path matches the design. The THredis profile matches the design. On GET at eight threads, `workerThreadMain` accounts for 17.0% of CPU – the worker’s main loop accounting for the largest share of CPU time, as intended. The shard dictionary lookup path (`dictFindLinkInternal`, `kvGetKey`) accounts for ~9.6% combined; the prefetch hash (`siphash`) for 3.88%; the reply buffer assembly (`_addReplyToBufferOrList`) for 6.31%; the IO-thread drain (`handleWorkerReplies`) for 0.32–2.56% across workloads; and the dispatch path (`canDispatchToWorker`) for 1.40%. Every design component named in §4.4–4.6 appears in the profile, confirming that the implementation realizes the design described earlier.

7.9.5 Cross-architecture observation. DragonflyDB and THredis target the same design cost – per-request kernel overhead – through different reasons. DragonflyDB’s per-Proactor `io_uring` path tries to reduce syscall cost via batched submission and completion queues, but the per-completion kernel work (TCP, netfilter, SRSO mitigations) remains per-request. THredis’s consolidated drain (§4.6) explicitly reduces send-side syscalls to one per pipeline batch, trading `io_uring`’s general-purpose batching for pipeline-specific consolidation; recent DBMS work similarly finds that effective `io_uring` use depends on architectural integration rather than simple API replacement [13]. The data shows the consolidation has more leverage on small-payload workloads: THredis spends 27.6% of GET cycles in the kernel, DragonflyDB 68.5%.

7.10 Summary

In the headline pipelined setting, THredis delivers a 1.60–1.98× throughput improvement over stable Redis on small-payload 1:1 workloads at 8 and 10 server threads, with comparable p99 latency at the main 8-thread point but higher p99 at the oversubscribed 10-thread point. On read-heavy 1:10 workloads it delivers a 1.26–1.63× improvement at 8 threads and above, scaling usefully into shared-SMT and oversubscribed configurations where stable Redis regresses; at 4 threads, where each worker has only ~2 cores behind it, stable Redis edges out THredis on the read-heavy mix. The payload-size sweep extends the same picture across data sizes: at pipeline 16 and above on `redis-benchmark`, THredis leads stable Redis on every GET and SET measurement at both 256 B and 1024 B by 1.29–1.50×, and stays ahead of KeyDB and DragonflyDB throughout. The win is conditional on pipelining: THredis loses at pipeline depth 1 and is 14–31% slower than Redis on synchronous YCSB workloads, where there is no batch to amortize the dispatch hop or to consolidate syscalls across.

At the implementation level, THredis pays approximately 1.5× the per-request cycle cost of stable Redis but executes those cycles in parallel across worker threads that stable Redis’s design cannot usefully use. The implementation’s hot path matches the design description: `workerThreadMain`, the prefetch and SipHash steps,



Top hot function per (server, workload), with % of CPU

	GET	SET	HASH	ZSET	BITMAP
Redis	<code>_copy_from_iter</code> 17.5%	<code>processInputBuffer</code> 5.0%	<code>_addReplyToBufferOrList.part.0</code> 19.2%	<code>_addReplyPayloadToList.lto_priv.0</code> 16.5%	<code>prefetchCommands</code> 5.4%
KeyDB	<code>fastlock_lock</code> 77.2%	<code>fastlock_lock</code> 76.0%	<code>fastlock_lock</code> 84.6%	<code>fastlock_lock</code> 77.1%	<code>fastlock_lock</code> 59.4%
DragonflyDB	<code>dfly::detail::ascii_unpack</code> 3.3%	<code>PrimeTablePolicy>::iterator<false_</code> 4.6%	<code>dfly</code> 16.9%	<code>dfly</code> 7.9%	<code>0x00000000000000c48</code> 3.5%
THredis	<code>workerThreadMain</code> 17.0%	<code>workerThreadMain</code> 17.7%	<code>_addReplyToBufferOrList.part.0</code> 15.8%	<code>_addReplyPayloadToList.lto_priv.0</code> 11.5%	<code>workerThreadMain</code> 40.5%

(b) Top hot function per (server, workload) cell. Cell intensity encodes the function's share of CPU. The four servers' bottleneck patterns are visually distinct.

Figure 13: Cycle breakdown at eight server threads. The top panel separates userspace and kernel time; the bottom panel shows the hottest function in each server/workload cell.

the SPSC dispatch path, and the consolidated drain all appear in the profile as expected.

8 Discussion

The evaluation in Section 7 answers the five questions posed at its opening, but the results admit more than one interpretation. This section takes the position that the headline numbers are correct but easy to misread, and that the more useful contribution of THredis is what its profile tells us about *where* a Redis-compatible server can usefully exploit additional cores. We organize the discussion around four observations – the conditions under which execution-level pipelining pays off, the cost it imposes when those conditions are absent, the architectural comparison the cycle breakdown makes possible, and the limitations of the current design – followed by the future-work directions those limitations suggest.

8.1 When execution-level pipelining pays off

The headline result – a 1.60–1.98× throughput improvement over stable Redis at 8 and 10 server threads on the 1:1 mix – holds at the intersection of three conditions: pipelined clients, an IO/worker split matched to where the per-request bottleneck sits, and single-key commands that fit the fast path. Each condition can be read off the data directly.

8.1.1 Pipelining is required, not merely useful. The pipeline-depth crossover in §7.4 is sharp: at depth 1, THredis is slower than stable Redis on both mixes (0.62 M vs 0.85 M on 1:1), and the gap only inverts once depth reaches 16 or higher. The synchronous YCSB result repeats the same finding at the application layer: THredis is 14–31% behind stable Redis on workloads A–C at 200 client threads, because YCSB's Redis binding does not pipeline. This is consistent with the design's underlying premise rather than a surprise. Execution-level pipelining amortizes the dispatch hop across the

batch of independent in-flight commands; without a batch, the hop is pure overhead.

The execution-side amortization is not the only contributor. The consolidated reply drain (§4.6) amortizes *syscall* cost across the same batch: replies for an entire pipeline batch are spliced into the real client's output buffer and emitted in a single *sendmsg* per drain pass, reducing wire-side syscall count to roughly $1/P$ of a per-request design. The cycle-attribution data in §7.9.1 shows the size of this effect indirectly: the GET kernel-time fraction drops from 55.3% on stable Redis to 27.6% on THredis, with the syscall consolidation responsible for a substantial portion of that reduction. Without the batch flush, the per-command socket-write overhead would offset much of the parallel-execution gain at high pipeline depths. The symmetric optimization on the read side – parsing the entire pipeline batch in one pass rather than one command at a time – has not been implemented; we discuss it as future work in §8.5.2.

8.1.2 The IO/worker split has to match per-request cost. The IO/worker budget data in §7.6.3 shows that the right split depends on where per-request work sits, not on payload size. Short string commands like GET and SET prefer IO-heavy splits because per-request execution is cheap and the worker drains its queue before the IO thread has parsed the next batch: $4i+2w$ beats $2i+4w$ by 28–32% at six server threads. Compute-heavy commands flip the bottleneck to the worker side, and the preferred split flips with it: HGETALL prefers $2i+4w$ over $4i+2w$ by 1.56×, ZRANGEBYSCORE by 1.61×. Bandwidth-bound BITCOUNT on 1 MiB values returns to favoring IO threads ($4i+2w$ by 1.75×) because the network egress path—assembling and emitting large reply buffers—is mostly an IO-side cost.

The unifying rule is to tune the split toward whichever side carries higher per-request cost. For deployments dominated by short cache commands, the headline $4i+4w$ configuration is well-matched and delivers consistent wins across payload sizes (§7.5.1). For deployments dominated by heavier per-request work—hash aggregations, sorted-set range queries—a worker-heavier split is correct, and the gap between the right and wrong static configuration can reach 50–69% throughput (§7.6.3). At total thread budgets of 8 or above the split sensitivity narrows substantially, but a heterogeneous command mix—or one whose mix shifts over the day—will still be sub-optimal at any single static configuration. This motivates automatic split tuning, which we discuss in §8.5.

8.1.3 Single-key commands are the fast path. The data-structure sweep in §?? shows the same pattern across command classes. THredis's largest gain (2.59×) comes on HGETALL where each command has substantial CPU work and addresses one key. The one place THredis is clearly beaten in this sweep is ZRANGEBYSCORE, where DragonflyDB wins by 9%. The performance analysis in §5.5 predicted exactly this outcome: THredis's fast path is single-key, and DragonflyDB's future/fiber model [10] is more general at the cost of higher fixed coordination overhead per command.

These three conditions together identify a workload class – pipelined, CPU-bound, single-key cache traffic – rather than a universal speedup. For the production cache and session-store workloads described in §2.1, this class is dominant, which is why we believe the design point is useful in practice. For workloads outside

this class, the right answer may be a different server, not a different configuration.

8.2 The cost of the design when pipelining is absent

A design that loses 14–31% on a research-standard benchmark (YCSB) [8] requires explanation, not only acknowledgement. The cost has three parts that share a common origin: pipelining is what makes THredis's overheads worthwhile, and synchronous operation removes that amortization. Two of the three are added overheads that pipelined operation hides through parallelism – the per-command dispatch hop and the cache-coherence traffic of cross-thread handoff. The third is a benefit unrealized: the consolidated drain's syscall reduction depends on having a batch to consolidate, and synchronous workloads have batches of one.

8.2.1 The dispatch hop. The intrinsic cost is the dispatch hop itself. Each command pays a hash to select a worker, an SPSC enqueue, a worker dequeue, a completion bit set, and a drain step on the IO thread. The hot-path analysis in §7.9 attributes about 1.40% of CPU to *canDispatchToWorker* and 0.32–2.56% to *handleWorkerReplies* across workloads – not large fractions, but real per-command work that stable Redis does not pay. At pipeline depth 1, this cost is paid once per client request rather than once per batch, so the dispatch hop is not amortized at all.

8.2.2 Cache-coherence traffic. The second intrinsic cost is cache-coherence traffic. The fake-client ring, the SPSC queue, and the reply mask all live on cache lines that move between the IO thread and the worker thread. The cycle-per-request data in §7.8.1 shows this directly: THredis executes within 5% of stable Redis's instruction count, but pays 1.4–1.9× the cycles. Almost all of the gap is memory-system stalls, not extra computation. Under pipelined load these stalls are hidden by the prefetch path described in §4.5; under synchronous load they are exposed because there is no later command on the same connection to overlap with.

8.2.3 Loss of batch-flush syscall savings. In pipelined operation, the consolidated reply drain (§4.6) emits one *sendmsg* per pipeline batch rather than one per command, scaling wire-side syscall count to roughly $1/P$ of a per-request design. §8.1.1 credited this as a substantial contributor to the GET kernel-time reduction over stable Redis. Without pipelining, the savings are zero by definition: each command produces one reply, the drain pass has exactly one slot to splice, and the syscall count returns to one per command – the same as stable Redis. The drain still runs but has no batch to consolidate. The dispatch-hop and cache-coherence costs above are paid as before; the effect that would have offset them in pipelined mode is gone. This is why YCSB exposes the worst case of THredis's design: the synchronous client model removes the one effect that pulls THredis's kernel cost below stable Redis's, while the design's extra per-command overheads remain. The in-order retirement tail discussed in §7.7.3 is a separate cost paid under saturated pipelined load rather than under synchronous load, and we return to it as a future-work target in §??.

8.3 What the cycle breakdown reveals about each architecture

The eval's cycle attribution puts each server's bottleneck in a recognizably different place; the four-way contrast is more informative than any single throughput number.

8.3.1 Stable Redis: kernel-bound on small reads. Most GET cost is in per-request socket-write syscalls, and the asymmetric IO-thread load shows that adding helpers beyond four does not relieve the bottleneck [27] (§7.9.1). THredis's consolidated drain attacks this same kernel cost from a different angle, batching syscalls per pipeline rather than per command.

8.3.2 KeyDB: dominated by lock acquisition. `fastlock_lock` costs several times what the command logic it protects costs (§7.9.2). The lock-based model preserves a broader command surface than THredis's single-writer partitioning [32], but the cost of that flexibility shows up directly in the cycle profile rather than only on contended workloads.

8.3.3 DragonflyDB: per-completion kernel cost. DragonflyDB pays an outlier cycle cost on GET specifically – roughly 33 K cycles per request, against THredis's 7.8 K and stable Redis's 5.5 K (§7.8.1) – a 4–6× overhead that no other workload in the sweep replicates. The cost spreads across the TCP stack, netfilter, and microcode mitigations rather than concentrating in one function (§7.9.3) – the pattern of a design whose per-completion bookkeeping is not hidden by user-space work when per-request work is small [10]. This supports the broader claim from recent DBMS work [13] that `io_uring` pays off only when integrated with batching and architectural changes; THredis's consolidated drain is one such change, batching across the pipeline rather than per-Proactor.

8.3.4 THredis: profile matches the design. The THredis profile (§7.9.4) is the strongest evidence that the implementation does at runtime what the design in Sections 4.2–4.6 prescribes. Every named structural component appears among the hot symbols at roughly the proportion the design predicts.

Equally important is what does *not* appear. There is no contended lock symbol – the structural single-writer invariant eliminates lock contention by construction rather than by tuning. There is no large scheduler-overhead fraction, because workers spin briefly on PAUSE rather than blocking. The kernel fraction is much smaller than stable Redis's or DragonflyDB's, because the consolidated drain reduces per-batch syscall count to one `sendmsg`. The profile is not itself a contribution, but it establishes that no large unaccounted-for cost dominates the implementation: the throughput numbers reflect the design as described, not a hidden bottleneck.

8.3.5 Cross-architecture lesson. “Redis is slow on multicore” is too coarse a claim. Each baseline has a different bottleneck, and the right ranking depends on which dominates a given workload. THredis is faster on workloads where per-request syscall and execution serialization dominate and where the work fits the single-writer fast path; it is not universally faster.

8.4 Limitations

The current THredis design has limits that the evaluation has touched on but that should be stated together.

8.4.1 Single-key fast path only. The dispatch mechanism described in §4.4.2 assumes each command has one key that maps to one worker. Commands that touch multiple keys – MSET, MGET, SUNION-STORE, multi-key transactions, scripting with cross-key access – cannot use the same low-cost dispatch path. They must either fall back to a slower path that coordinates across workers, or be excluded from execution-level pipelining. The current implementation takes the second route for the commands not on the fast path, which is acceptable for the cache and session-store workloads we target but does not match the full Redis command surface.

8.4.2 No hot-key handling. The single-writer-per-key invariant from §4.8 is also the design's load-balancing limit. A workload where one key receives a disproportionate share of writes will bottleneck on the worker that owns that key; the other workers cannot help even if they are idle. The performance analysis in §5.3 discusses this; the evaluation does not test it directly because we did not measure adversarial skew. We expect the design to degrade gracefully with moderate Zipfian skew (which is implicit in the YCSB workloads and consistent with production key-value access patterns [2]) and to degrade sharply with adversarial single-hot-key writes.

8.4.3 Static IO/worker split. The IO/worker budget data in §7.6 shows that the best split depends on the workload's command mix, with up to 69% throughput between two splits at the same total thread count. The current implementation requires the operator to pick the split at startup. A workload with a heterogeneous command mix, or one whose mix changes over the day, will be sub-optimal at any static configuration. We discuss possible automatic tuning in §8.5.

8.4.4 Single-machine evaluation. All measurements are from one AMD Ryzen 7 7700X with 8 physical cores. Multi-socket NUMA effects, larger core counts, and cluster-mode deployments are not evaluated. The design's shard-per-worker model is amenable to NUMA-aware placement, but the evaluation does not show this empirically.

8.4.5 RDB persistence is not supported. Redis's RDB snapshot mechanism relies on `fork()` of a process whose key space is mutated by a single execution thread, so that the child can walk the entire dataset via copy-on-write without racing with ongoing command execution. THredis partitions the key space across worker threads, each owning its own shard, and a single coordinated fork would require a synchronization point at which all workers are quiescent simultaneously – something the current design does not provide. The implementation therefore does not support RDB at present. We discuss the natural remedy – per-worker background-save forks – in §8.5. Recent work on `io_uring`-based persistence in Redis [5] is complementary along a different axis (storage I/O rather than fork coordination) and could be combined with a per-worker persistence path in future work.

8.5 Future work

Five engineering directions follow naturally from the limitations and results above. A sixth, included for completeness, is more speculative.

8.5.1 Automatic IO/worker tuning. The split-sensitivity data in §7.6 suggests a control loop: observe the per-command queue depth on the IO and worker sides over a short window, and re-balance threads toward whichever side has the longer queue. Because threads are persistent OS-level threads in the current design, full re-balancing requires either restart or thread-migration support; a more practical first step is to allow the operator to set the split per command class and have a dispatcher route commands accordingly. The 30–60% throughput swings on workloads where the static split is mismatched suggest the engineering effort is justified.

8.5.2 Batched pipeline parsing. The consolidated reply drain (§4.6) was a substantial contributor to the headline result by amortizing wire-side syscalls across the pipeline batch (§8.1.1). The symmetric optimization on the read side has not been implemented: the IO thread currently parses each pipelined command individually as it is read off the socket, dispatching it to a worker before parsing the next. A batched parser would bulk-decode the RESP protocol over the entire contiguous read buffer, populate a sequence of argv arrays in one pass, and dispatch them to their respective workers without reentering the read loop between commands. The benefits would mirror the drain's: shared parsing state, better instruction-cache locality, and a single dispatch loop rather than one per command. We have not measured this effect, but the magnitude of the drain's contribution to the GET kernel-time reduction (§7.9.1) suggests the symmetric optimization is worth investigating, particularly at high pipeline depths where the per-command IO-thread overhead becomes a larger fraction of total work.

8.5.3 Hot-key handling. A natural extension is to detect hot keys at dispatch time and either replicate them across workers (for read-heavy hotness) or split them into sub-keys at the storage layer (for write-heavy hotness). The first option preserves the single-writer invariant if reads are routed away from the writing worker; the second weakens the invariant in a controlled way. Replication-style hot-key handling has prior art in epoch-based concurrent stores such as FASTER [4], and partitioning-based hot-key routing is used in flow-steered designs such as MICA [19]. Either approach must measure hotness without making dispatch significantly more expensive than the current power-of-two AND.

8.5.4 Multi-key fast paths. For multi-key commands whose keys happen to map to the same worker, the current design could in principle execute them on the fast path; the implementation does not currently exploit this. For commands whose keys span multiple workers, a scatter-gather mechanism similar in spirit to DragonflyDB's future model [10] could coexist with the single-key fast path, paying the higher coordination cost only for commands that need it.

8.5.5 Per-worker RDB persistence. The natural fit between THredis's shard-per-worker partitioning and Redis's fork-based snapshot mechanism [30] is to have each worker launch its own background-save fork over its private shard, producing a per-shard snapshot

file rather than a single global one. Because each worker is the sole writer to its shard, the fork can be taken at any moment between commands on the worker's own queue, with no cross-worker coordination required. The resulting collection of per-shard files together represents the database state, and recovery loads each file into the corresponding worker. Beyond restoring parity with stable Redis, this approach has the potential to *outperform* stable Redis on both save and load. Stable Redis's RDB save is single-process: one forked child walks the entire keyspace sequentially. With per-worker forks, the walk parallelizes naturally across W shards, with each child writing its own file. RDB load benefits more straightforwardly: stable Redis parses the snapshot serially on a single thread, while per-shard files can be parsed and inserted concurrently by their owning workers, with no contention since each worker writes only into its own shard's dictionary. We have not measured these effects; the engineering surface includes the snapshot file format, the recovery-time worker-to-file binding, and the interaction with replication's full-resync path, but the underlying design premise – single-writer ownership of each shard – is already in place.

9 Conclusion

THredis adds execution-level pipelining to a Redis-compatible server by splitting work across two specialized thread pools: IO threads that own client connections and parse incoming requests, and worker threads that own disjoint shards of the keyspace and execute commands in parallel. This multi-threaded structure is what enables execution-level pipelining in the first place – a single-execution-thread design like stable Redis can pipeline commands on the wire but must still process them serially in user space, while THredis can have many commands from the same client connection in flight simultaneously across different workers. Three mechanisms make the architecture viable for Redis-compatible workloads: a fake-client abstraction that lets the existing `void(*proc)(client*)` command-handler API run on a worker thread without modification, a structural single-writer-per-key partition that eliminates keyspace data hazards by ownership rather than by locking, and an in-order retirement drain that emits replies to the socket in dispatch order while allowing workers to complete out of order. Dispatch and completion are lock-free throughout, using one SPSC queue per producer-consumer pair and an atomic per-client reply-ready bitmask. The structure corresponds, deliberately, to Tomasulo's algorithm for out-of-order execution with in-order commit [36]; the analogy is useful as scaffolding but is not the contribution. The contribution is the specific multi-threaded realization that makes this model viable for a Redis-compatible command surface while preserving the client-visible reply-ordering and same-key semantics that existing Redis applications depend on.

On pipelined single-key workloads at a thread split matched to the bottleneck, this combination delivers a 1.60–1.98× throughput advantage over stable Redis at 8 and 10 server threads on the balanced mix, and 1.26–1.63× on the read-heavy mix, with median latency that is consistently lower than all three baselines and p99 latency that is competitive with Redis at the non-oversubscribed operating point. The cycle breakdown shows that the gain is not free: THredis pays roughly 1.5× the per-request cycle cost of stable

Redis, but those cycles execute in parallel across worker threads that stable Redis's design cannot usefully use.

Outside this regime, THredis is the wrong tool. At pipeline depth 1, on synchronous YCSB workloads, and on multi-key commands, the design either loses ground to stable Redis or falls behind DragonflyDB. These are the intrinsic loss cases: pipeline depth 1 pays the dispatch hop without a batch to amortize over, synchronous YCSB removes both the parallel execution amortization and the consolidated drain's syscall savings, and multi-key commands fall outside the single-key fast path entirely. Once clients pipeline, the win is consistent: redis-benchmark shows THredis ahead of stable Redis on every GET and SET measurement at 256 B and 1024 B by 1.29–1.50× at pipelines 16 and 32, and ahead of KeyDB and DragonflyDB on the same cells.

For the cache, session-store, and short-command workloads that motivate much of the current Redis deployment [2, 26], the dominant costs are two: execution serialization in user space, and per-request syscall overhead on the wire side. THredis addresses both. Execution-level pipelining with structural single-writer partitioning parallelizes command execution across worker threads, exploiting cores that stable Redis's design cannot usefully use. The consolidated reply drain – enabled by having multiple pipelined commands complete on a single client connection at the same time – amortizes wire-side syscalls across the batch, dropping the GET kernel-time fraction from 55.3% on stable Redis to 27.6% on THredis. Both contributions arise from the same multi-threaded architecture; neither is possible without separating IO threads from worker threads. The remaining engineering surface – batched pipeline parsing as the symmetric read-side counterpart, automatic IO/worker tuning, hot-key handling, multi-key fast paths, per-worker RDB persistence, and NUMA-aware shard placement – is, we believe, tractable from the design point established here, and constitutes the natural next step for a Redis-compatible server that takes execution-level pipelining as its starting assumption.

10 Use of AI

All algorithmic and architectural ideas in this project are our own and were not generated or influenced by AI systems. The core design decisions behind THredis, including execution-level pipelining, the fake-client abstraction, worker-thread and IO-thread separation, batch flushing, correctness requirements, benchmarking methodology, and the comparison to Tomasulo-style execution, were defined by the authors.

AI tools were used as implementation and writing assistants. The initial implementation of the fake client, worker thread, IO thread, and batch-flush mechanisms was written mostly by hand. During this process, Claude was used for guided navigation, debugging, and consultation on the Redis code base, which is large and difficult to modify safely. Implementations specifically related to xxHash-based dispatch and software prefetching were written with substantial AI assistance, although the ideas and motivation for these optimizations were our own.

Claude was also instructed to add defensive guards where appropriate, based on instability observed during correctness testing and extensive benchmarking. These guards were added to improve robustness after crashes and failures encountered in earlier versions

of the system. All results presented in this paper were collected only after the implementation had been made significantly more stable.

The benchmark plan was designed by the authors. This includes the choice of benchmarks, the configurations to sweep, the thread-count studies, pipeline-depth studies, payload-size studies, read/write-ratio experiments, and the systems selected for comparison. However, the automation scripts used to run the benchmark sweeps, including the autobench shell script and the Python analysis scripts, were written with AI assistance.

The paper itself was also written with a large amount of AI assistance due to the scale of the project and the number of experimental results. However, the paper structure was defined by the authors. The implementation details were specified by the authors, while AI assistance was used primarily for phrasing and drafting. All citations, analogies, and connections to Tomasulo's algorithm were identified and directed by the authors. The background research, conclusions, analysis, comparisons, and interpretation of what we observed about THredis and the other systems were also performed by the authors. In particular, the claims about what the system does, what the experiments show, and what conclusions should be drawn from those results reflect the authors' own understanding and judgment.

11 Publishability and Research Significance

We believe this project has strong potential for publication as a systems research paper. One of the authors has previously published in this research area through the Springer LNCS CLOUD conference, and based on that prior experience, our judgment is that THredis is substantially more novel, technically ambitious, and publication-oriented than a typical course project.

The main reason is that THredis does not simply tune parameters or optimize an isolated bottleneck in Redis. Instead, it introduces a different execution model for Redis-compatible key-value stores: execution-level pipelining. To the best of our knowledge from our background research, existing Redis-compatible systems treat pipelining primarily as a wire-level batching mechanism. Redis executes commands serially on the main execution thread; KeyDB adds parallelism across server threads but relies on shared-key-space synchronization; and DragonflyDB uses a sharded/fiber-based architecture but does not expose the same per-connection execution-level pipelining model. THredis instead separates IO ownership from command execution, dispatches commands from the same pipelined connection to worker threads, and retires replies in dispatch order. This combination of fake clients, sharded workers, SPSC dispatch queues, reply-ready masks, and in-order drain gives Redis-style pipelining a server-side execution meaning that we have not seen in prior Redis-compatible systems.

We also believe the performance result is meaningful for a practically important workload. Pipelined small-payload key-value access is likely one of the most important use cases for Redis-like servers: it appears in caching, session storage, counters, repeated reads, and latency-sensitive backend services. In this regime, THredis achieves large throughput gains over stable Redis while preserving client-visible reply ordering and same-key semantics. The gains are not presented as universal; our report also identifies cases where

the design does not help, including pipeline depth 1, synchronous YCSB clients, bandwidth-bound large payloads, and workloads limited by hot-key serialization. However, under the workload that the system is explicitly designed for, the improvement is substantial.

The work also includes several components expected of a publishable systems artifact: a working Redis-derived implementation, a new parallel architecture, correctness reasoning, stability fixes based on stress testing, comparisons against multiple Redis-compatible baselines, benchmark sweeps across thread count, pipeline depth, payload size, command type, and read/write ratio, and profiling-based analysis explaining where the performance differences come from. We also submit the benchmark automation and analysis scripts as deliverables to support reproducibility.

At the current stage, we view THredis as a strong research prototype rather than a fully polished submission artifact. The remaining work is primarily engineering and experimental strengthening rather than a lack of research novelty. With additional time, we would further stabilize and clean up the implementation, run the full benchmark suite on dedicated server-class hardware, repeat all major experiments with multiple trials such as $N = 5$ sweeps, report confidence intervals or variance, and package the artifact so that external reviewers can reproduce the results more easily. We have discussed this direction with the instructor, and our understanding is that continued work beyond the class timeline is acceptable if the project needs additional polishing before being considered publication-ready.

Our long-term goal is to develop this work into a submission suitable for a major systems or data-management venue, with VLDB as a particular dream target. We recognize that a VLDB-quality submission would require additional polishing beyond the current class report, especially stronger artifact packaging, repeated trials, broader evaluation on server-class hardware, and a more rigorous positioning against database and key-value store systems literature.

For these reasons, we believe THredis is not merely a class implementation project, but a credible systems research prototype. With additional polishing, broader artifact packaging, repeated benchmark sweeps, and validation on stronger server hardware, we believe it could be developed into a publishable paper.

References

- [1] Dany Almeida, Maria Lopes, Luzia Saraiva, Maryam Abbasi, Pedro Martins, José Silva, and Paulo Váz. 2023. Performance Comparison of Redis, Memcached, MySQL, and PostgreSQL: A Study on Key-Value and Relational Databases. In *2023 Second International Conference On Smart Technologies For Smart Nation (SmartTechCon)*. 902–907. doi:10.1109/SmartTechCon57526.2023.10391649
- [2] Berk Atikoglu, Yuehai Xu, Eitan Frachtenberg, Song Jiang, and Mike Paleczny. 2012. Workload analysis of a large-scale key-value store. In *Proceedings of the 12th ACM SIGMETRICS/PERFORMANCE Joint International Conference on Measurement and Modeling of Computer Systems* (London, England, UK) (*SIGMETRICS '12*). Association for Computing Machinery, New York, NY, USA, 53–64. doi:10.1145/2254756.2254766
- [3] Jean-Philippe Aumasson and Daniel J. Bernstein. 2012. SipHash: A Fast Short-Input PRF. In *Progress in Cryptology - INDOCRYPT 2012*, Steven Galbraith and Mridul Nandi (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 489–508.
- [4] Badrish Chandramouli, Guna Prasaad, Donald Kossmann, Justin Levandoski, James Hunter, and Mike Barnett. 2018. FASTER: A Concurrent Key-Value Store with In-Place Updates. In *Proceedings of the 2018 ACM International Conference on Management of Data (SIGMOD)*. ACM, Houston, TX, USA, 275–290. <https://www.microsoft.com/en-us/research/wp-content/uploads/2018/03/faster-sigmod18.pdf>
- [5] Le-Gao Chen, Yanzhi Li, Tipporn Laohakangvalvit, and Midori Sugaya. 2025. Asynchronous I/O Persistence for In-Memory Database Servers: Leveraging io_uring to Optimize Redis Persistence. In *CLOUD Computing – CLOUD 2024*, Yang Wang and Liang-Jie Zhang (Eds.). Springer Nature Switzerland, Cham, 11–20.
- [6] Xing Chen, Feijie Wang, Jinmei Xu, Dongsheng Zhu, Ping Tan, and Jien Ma. 2020. A distributed cache system based on Redis for high-speed railway catenary monitoring system. In *2020 Chinese Automation Congress (CAC)*. 2048–2053. doi:10.1109/CAC51589.2020.9326531
- [7] Yann Collet. [n. d.]. xxHash – Extremely fast non-cryptographic hash algorithm. <https://github.com/Cyan4973/xxHash>. Accessed: 2026-04-28.
- [8] Brian F. Cooper, Adam Silberstein, Erwin Tam, Raghu Ramakrishnan, and Russell Sears. 2010. Benchmarking cloud serving systems with YCSB. In *Proceedings of the 1st ACM Symposium on Cloud Computing (Indianapolis, Indiana, USA) (SoCC '10)*. Association for Computing Machinery, New York, NY, USA, 143–154. doi:10.1145/1807128.1807152
- [9] Rohan Doshi, Sanika Inamdar, Tanmay Karmarkar, and Madhuri Wakode. 2024. Distributed MQTT Broker: A Load-Balanced Redis-Based Architecture. In *2024 International Conference on Emerging Smart Computing and Informatics (ESCI)*. 1–6. doi:10.1109/ESCI59607.2024.10497427
- [10] DragonflyDB Inc. 2024. Dragonfly Shared-Nothing Architecture. <https://github.com/dragonflydb/dragonfly>.
- [11] DragonflyDB Inc. 2024. DragonflyDB Documentation. <https://www.dragonflydb.io/docs>.
- [12] Jingqi Huo. 2023. Anomaly Time Series Detection and Prediction for Internet of Things (IoT) Based on Redis Visualization and Alert. In *2023 International Conference on Industrial IoT, Big Data and Supply Chain (IIoTBDSC)*. 1–7. doi:10.1109/IIoTBDSC60298.2023.00027
- [13] Matthias Jasny, Muhammad El-Hindi, Tobias Ziegler, Viktor Leis, and Carsten Binnig. 2026. io_uring for High-Performance DBMSs: When and How to Use It. *Proceedings of the VLDB Endowment* 19, 1 (2026), XXX–XXX. <https://arxiv.org/abs/2512.04859> arXiv:2512.04859v2.
- [14] Anuj Kalia, Michael Kaminsky, and David G. Andersen. 2014. Using RDMA Efficiently for Key-Value Services. In *Proceedings of the 2014 ACM Conference on SIGCOMM*. ACM, Chicago, IL, USA, 295–306. doi:10.1145/2619239.2626299
- [15] Sawroop Kaur and G. Geetha. 2020. SIMHAR - Smart Distributed Web Crawler for the Hidden Web Using SIM+Hash and Redis Server. *IEEE Access* 8 (2020), 117582–117592. doi:10.1109/ACCESS.2020.3004756
- [16] Leslie Lamport. 1977. Concurrent reading and writing. *Commun. ACM* 20, 11 (Nov. 1977), 806–811. doi:10.1145/359863.359878
- [17] Kunhui Li, Kun Guo, and Hong Guo. 2019. Financial Big Data Hot and Cold Separation Scheme Based on HBase and Redis. In *2019 IEEE Intl Conf on Parallel & Distributed Processing with Applications, Big Data & Cloud Computing, Sustainable Computing & Communications, Social Computing & Networking (ISPA/BDCLOUD/SocialCom/SustainCom)*. 1612–1617. doi:10.1109/ISPA-BDCLOUD-SustainCom-SocialCom48970.2019.00237
- [18] Songhuan Li, Hong Jiang, and Mingkan Shi. 2017. Redis-based web server cluster session maintaining technology. In *2017 13th International Conference on Natural Computation, Fuzzy Systems and Knowledge Discovery (ICNC-FSKD)*. 3065–3069. doi:10.1109/FSKD.2017.8393274
- [19] Hyeontaek Lim, Dongsu Han, David G. Andersen, and Michael Kaminsky. 2014. MICA: A Holistic Approach to Fast In-Memory Key-Value Storage. In *11th USENIX Symposium on Networked Systems Design and Implementation (NSDI '14)*. USENIX Association, Seattle, WA, USA, 429–444. <https://dl.acm.org/doi/10.5555/2616448.2616488>
- [20] Linux man-pages project. 2025. *epoll(7) – Linux manual page* (Linux man-pages 6.16 ed.). [man7.org. https://man7.org/linux/man-pages/man7/epoll.7.html](https://man7.org/linux/man-pages/man7/epoll.7.html) HTML rendering by Michael Kerrisk. Accessed: 2026-04-28.
- [21] Yandong Mao, Eddie Kohler, and Robert Morris. 2012. Cache Craftiness for Fast Multicore Key-Value Storage. In *Proceedings of the 7th ACM European Conference on Computer Systems (EuroSys)*. ACM, Bern, Switzerland, 183–196. <https://pdos.csail.mit.edu/papers/mas/tree/eurosys12.pdf>
- [22] Gulara Muradova, Mehran Hematyar, and Jala Jamalova. 2022. Advantages of Redis in-memory database to efficiently search for healthcare medical supplies using geospatial data. In *2022 IEEE 16th International Conference on Application of Information and Communication Technologies (AICT)*. 1–5. doi:10.1109/AICT55583.2022.10013544
- [23] Marius Petrescu et al. 2025. Comparative Performance Analysis of Modern NoSQL Data Technologies: Redis, Aerospike, and Dragonfly. *arXiv preprint arXiv:2510.08863* (2025). <https://arxiv.org/abs/2510.08863>
- [24] Redis. [n. d.]. LLM Caching with RedisVL. https://redis.io/docs/latest/develop/ai/redisvl/user_guide/llmcache/. Accessed: 2026-04-28.
- [25] Redis. [n. d.]. Redis for AI. <https://redis.io/redis-for-ai/>. Accessed: 2026-04-28.
- [26] Redis, Inc. [n. d.]. Redis Use Cases and Industry Adoption. <https://redis.io/>. Accessed: 2026-04-23.
- [27] Redis Ltd. 2024. Redis 6 I/O Threads. https://redis.io/docs/latest/operate/oss_and_stack/management/optimization/io-threads/.
- [28] Redis Ltd. 2024. Redis Data Types. <https://redis.io/docs/data-types/>.
- [29] Redis Ltd. 2024. Redis Documentation. <https://redis.io/docs/>.
- [30] Redis Ltd. 2024. Redis Internals and Event Loop. <https://redis.io/docs/reference/internals/>.

- [31] Nadia Ben Seghier and Okba Kazar. 2021. Performance Benchmarking and Comparison of NoSQL Databases: Redis vs MongoDB vs Cassandra Using YCSB Tool. In *2021 International Conference on Recent Advances in Mathematics and Informatics (ICRAMI)*. 1–6. doi:10.1109/ICRAMI52622.2021.9585956
- [32] Snap Inc. 2024. KeyDB Documentation. <https://docs.keydb.dev/>.
- [33] Stack Overflow Meta. 2015. How Redis is Used in Stack Overflow? <https://meta.stackoverflow.com/questions/306599/how-redis-is-used-in-stack-overflow>. Accessed: 2026-04-23.
- [34] M. Thomson and C. Benfield. 2022. HTTP/2. RFC 9113. doi:10.17487/RFC9113
- [35] Sruthii Thukkani. 2025. Scalable Vehicle Location Tracking Using Geohash and Redis. In *2025 International Conference on Inventive Computation Technologies (ICICT)*. 1673–1680. doi:10.1109/ICICT64420.2025.11005141
- [36] R. M. Tomasulo. 1967. An Efficient Algorithm for Exploiting Multiple Arithmetic Units. *IBM Journal of Research and Development* 11, 1 (1967), 25–33. doi:10.1147/rd.111.0025
- [37] Stephen Tu, Wenting Zheng, Eddie Kohler, Barbara Liskov, and Samuel Madden. 2013. Speedy Transactions in Multicore In-Memory Databases. In *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles (SOSP)*. ACM, Farmington, PA, USA, 18–32. <https://wzheng.github.io/silo.pdf>
- [38] VMware Tanzu. 2013. Case Study: Staple Yourself to a Tweet to Understand 30 Billion Redis Updates Per Day. <https://blogs.vmware.com/tanzu/case-study-staple-yourself-to-a-tweet-to-understand-30-billion-redis-updates-per-day/>. Accessed: 2026-04-23.
- [39] Xingbo Wu, Xiang Long, and Lei Wang. 2013. Optimizing Event Polling for Network-Intensive Applications: A Case Study on Redis. In *2013 International Conference on Parallel and Distributed Systems*. 687–692. doi:10.1109/ICPADS.2013.122
- [40] Peng Zhang, Lichao Xing, Ninggou Yang, Guolin Tan, Qingyun Liu, and Chuang Zhang. 2018. Redis++: A High Performance In-Memory Database Based on Segmented Memory Management and Two-Level Hash Index. In *2018 IEEE Intl Conf on Parallel & Distributed Processing with Applications, Ubiquitous Computing & Communications, Big Data & Cloud Computing, Social Computing & Networking, Sustainable Computing & Communications (ISPA/IUCC/BDCloud/SocialCom/SustainCom)*. 840–847. doi:10.1109/BDCloud.2018.00125